Contents

pyqula user guide

pyqula computes electronic structure of tight-binding models on lattices: band structures, densities of states and spectral functions, self-consistent (mean-field) interacting Hamiltonians, superconductivity, topological invariants, response functions, quantum transport, and classical spin and lattice-gas models.

Almost everything in this guide follows the same four steps – build a geometry, get its Hamiltonian, add terms to it, then ask the Hamiltonian for an observable:

```
from pyqula import geometry
g = geometry.honeycomb_lattice()    # 1. a lattice
h = g.get_hamiltonian()             # 2. its tight-binding Hamiltonian
h.add_zeeman([0.,0.,0.3])           # 3. add terms (in place)
(k,e) = h.get_bands()               # 4. compute an observable
```

Install with `pip install pyqula`, or from a clone of the repository with `pip install -e .` from its root. Note that `import pyqula` on its own exposes nothing – always import the submodule you need (`from pyqula import geometry`).

Each code block below carries its own imports and runs on its own, except where a block picks up the variables of the one before it inside the same section (a second call on an `h` that was just built, say).

The snippets here stop at the computed arrays and leave the plotting to you. Two other places in the repository take it further:

- `examples/` holds several hundred runnable scripts organized by dimensionality (`0d/ 1d/ 2d/ 3d/`, plus `transport/`, `embedding/`, `wannier/`, `classicalspin/`, `latticegas/`), most of them ending in a figure. Most sections below point at the relevant ones
- `jupyter-notebooks/functionalities/` holds 53 executed notebooks, one per feature, grouped the same way as the README's functionality list (single-particle Hamiltonians, mean field, topology, spectral functions, KPM, Wannierization, transport). Each carries its physics discussion and its output plots inline, so they are the place to look for what a result should actually look like

The last chapter, Main functions and methods, is a reference of the `Geometry`/`Hamiltonian` methods and their arguments.

Contents

- Electronic structure folding and unfolding
- Surface spectral functions
- Twisted bilayer graphene structural relaxation
- Topological insulators
- Response functions
- Quantum transport
- Single defects in infinite systems
- Wannierization
- Chebyshev kernel polynomial (KPM) methods
- Classical spin models
- Lattice gas models
- Ising models
- Main functions and methods

Setting up a Hamiltonian

In this basic tutorial we will address how to compute the band structure of a one dimensional tight binding model.

The Hamiltonian of a one dimensional tight binding chain takes the form

$$H = \sum_n c_n^\dagger c_{n+1} + h.c.$$

This model can be diagonalized analytically, giving rise to a diagonal Hamiltonian of the form

$$H = \sum_k \epsilon_k \Psi_k^\dagger \Psi_k$$

where energy momentum dispersion takes the form

$$\epsilon_k = 2\cos k$$

With the pyqula library, the previous band structure can be computed as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure
```

Including second and third neighbor hopping

By default, the Hamiltonian generated includes only first neighbor hopping $t_1 = 1$. However, we may want to consider a generalized Hamiltonian of the form

$$H = \sum_n c_n^\dagger c_{n+1} + t_2 \sum_n c_n^\dagger c_{n+2} + t_3 \sum_n c_n^\dagger c_{n+3} + h.c.$$

To compute the eigenvalues in this generalized model taking $t_2 = 0.2$ and $t_3 = 0.3$, we write

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian(tij=[1.0,0.2,0.3]) # Hamiltonian with t1,t2,t3
(k,e) = h.get_bands() # compute band structure
```

Including an onsite energy

The Hamiltonian can have an onsite energy term, that is equivalent to a chemical potential that takes the form

$$H = \mu \sum_n c_n^\dagger c_n$$

This can be added to the Hamiltonian as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
mu = 0.3 # value of the onsite
h.add_onsite(mu) # add onsite energy
```

Possible inputs

- Float: the same onsite energy is added to all the sites
- Iterable (list or array): adds a different onsite energy to each site in teh geometry
- Callable (function): adds a different onsite energy to each site according to its location **r**

Including an external Zeeman field

In the following we will consider that we want to add an external Zeeman field to the electronic system. We now include the existence of a spin degree of freedom, considering the Hamiltonian

$$H = H_0 + H_Z$$

where H_0 is the original tight binding Hamiltonian

$$H_0 = \sum_{n,s} c^\dagger_{n,s} c_{n+1,s} + h.c.$$

and

$$H_Z = \sum_{n,s,s'} \vec{B} \cdot \vec{\sigma}^{s,s'} c^\dagger_{n,s} c_{n,s'}$$

with n running over the sites and s, s' running over the spin degree of freedom. The magnetic field takes the form $\vec{B} = (B_x, B_y, B_z)$, and σ_α are the spin Pauli matrices. To add a magnetic field of the form $\vec{B} = (0.1, 0.2, 0.3)$ to our chain we write

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_zeeman([0.1,0.2,0.3]) # add the Zeeman field (modifies h in place)
(k,e) = h.get_bands() # compute band structure
```

Including an external orbital field

An external magnetic field can be included using the Peierls substitution

$$t_{\alpha\beta} \to t_{\alpha\beta} e^{i \int_{r_\alpha}^{r_\beta} \vec{A} \cdot d\vec{l}}$$

where $\vec{A}$ is the magnetic potential so that $\vec{B} = \nabla \times \vec{A}$. It can be used as shown in the example below

```
from pyqula import geometry
N = 20 # number of unit cells as the width
g = geometry.square_ribbon(N) # ribbon
h = g.get_hamiltonian() # generate the Hamiltonian
B = 0.02 # magnetic field in quantum flux unit
h.add_orbital_magnetic_field(B) # add an out-of plane magnetic field
(k,e) = h.get_bands() # compute the Landau-level band structure
```

Setting a filling

If you want to enforce a certain filling ν in a Hamiltonian, so that

$$\langle c^\dagger_n c_n \rangle = \nu$$

use

```
from pyqula import geometry
g = geometry.chain() # chain
h = g.get_hamiltonian()
h.set_filling(0.7) # enforce a filling
```

Possible inputs

- float: enforce the filling on average
- array: enforce that each site has a specific filling

Observables

Electronic band structures

For any system that is periodic in space, can compute the electronic band structure as given by

$$H = \sum_{k,\alpha} \epsilon_{k,\alpha} \Psi^\dagger_{k,\alpha} \Psi_{k,\alpha}$$

where α is the band index.

The previous calculation can be performed as

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure
```

Optional arguments - `kpath`: k-path to use, either a list of high-symmetry labels (e.g. `["G","K","M"]`) or explicit k-vectors; defaults to the geometry's standard path - `nk`: number of k-points along the path - `operator`: color/weight each band by the expectation value of an operator (or a list of operators), returning `(k,e,c)` instead of `(k,e)` - `num_bands`: for large sparse Hamiltonians, only compute this many bands around `central_energy` with ARPACK, instead of the full spectrum

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
(k,e,c) = h.get_bands(operator="velocity") # bands colored by group velocity
```

Passing a list of operators computes the expectation value of each of them for every eigenstate, returning one extra column per operator (`(k,e,c1,c2,...)` instead of `(k,e,c)`)

```
(k,e,c_sz,c_site) = h.get_bands(operator=["sz","site"]) # two operators at once
```

For a very large system (e.g. a moire supercell), diagonalizing the full Hamiltonian at every k-point is wasteful if only a handful of bands around the Fermi level are of interest. Passing `num_bands` switches to a sparse ARPACK solver that targets only those bands

```
(k,e) = h.get_bands(num_bands=20) # only the 20 bands closest to central_energy
```

See `examples/2d/velocity_bands/main.py` and `examples/2d/strain_TBG/main.py` for runnable versions.

Density of states

The density of states counts how many states are in a certain energy window. It is defined as

$$D(\omega) = \int \delta(\omega - \epsilon_k) dk$$

where ϵ_k are the eigenenergies of the Hamiltonian. It can be used as shown below

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
(es,ds) = h.get_dos()
```

Optional arguments - energies: array with the energies for which the DOS is computed - delta: smearing of the DOS - operator: operator to which the DOS is projected - mode: how the DOS is computed – `"ED"` (default, broadens a k-mesh band structure), `"Green"` (sums a Green's function per energy, useful when only a handful of energies are needed), `"KPM"` (Chebyshev kernel-polynomial expansion, for large sparse systems – see the "Chebyshev kernel polynomial (KPM) methods" section), or `"adaptive"` - nk: number of k-points in the mesh (`"ED"`/`"KPM"` modes)

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(tij=[0.5,0.,0.,0.5],has_spin=True)
h.add_rashba(0.7)
energies = np.linspace(-4.,4.0,60)
(e1,d1) = h.get_dos(energies=energies,delta=1e-2,mode="ED",nk=1000)
(e2,d2) = h.get_dos(energies=energies,delta=1e-2,mode="Green")
```

An operator can be passed to project the DOS onto a subspace, e.g. the sublattice-resolved DOS of a gapped honeycomb lattice

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
```

```
h = g.get_hamiltonian(has_spin=True)
h.add_sublattice_imbalance(.4)
(es,ds) = h.get_dos(operator="sublattice",nk=40,delta=5e-2)
```

See `examples/1d/dos_GF/main.py` and `examples/2d/operator_dos/main.py` for runnable versions.

Local density of states

The local density of states resolves the density of states by site: it counts how many states are in a certain energy window, weighted by how much of each state sits on site n. It is defined as

$$D(\omega, n) = \int \delta(\omega - \epsilon_k) |\langle \Psi_k | n \rangle|^2 dk$$

where ϵ_k are the eigenenergies of the Hamiltonian. It can be used as shown below

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
(x,y,d) = h.get_ldos()
```

Optional arguments - e: energy at which the LDOS is evaluated - delta: smearing of the LDOS - operator: operator to which the LDOS is projected - projection: `"TB"` (default, one value per lattice site), `"TBRS"` (same, but interpolated onto a continuous real-space map for smoother plotting), or `"atomic"` (projected onto atomic orbitals rather than tight-binding sites) - num_bands: for large sparse Hamiltonians, use ARPACK to only compute this many states around the target energy

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
(x,y,d) = h.get_ldos(e=0.0,projection="TBRS") # interpolated real-space map
```

`h.get_multildos()` computes the LDOS at many energies at once, writing one file per energy to a `MULTILDOS/` folder (useful for building an LDOS(x,y,E) movie/stack)

```
import numpy as np
h.get_multildos(energies=np.linspace(-2.0,2.0,100),projection="atomic")
```

See `examples/0d/island/main.py` (single-energy, `projection="TBRS"`, superconducting island) and `examples/readme_examples/ldos_island/main.py` (`get_multildos`, `projection="atomic"`) for runnable versions.

Momentum resolved spectral functions

Apart from the band structure, in certain cases it is interesting to compute the momentum resolved spectral function, that takes the form

$$A(k,\omega) = \delta(\omega - \epsilon_k)|\langle \Psi_k | A | \Psi_k \rangle|^2$$

where A is a certain operator. The previous quantity allows define a heatmap of the momentum-resolved spectral function. For the example, in a superconducting state, if operator is chosen to be projection onto the electron-sector, the previous quantity shows the electronic spectral fucntion

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
h.get_kdos_bands()
```

Optional arguments - energies: array with the energies for which the DOS is computed - delta: smearing of the DOS - operator: operator to which the DOS is projected

Fermi surfaces

For a 2D periodic Hamiltonian, `h.get_fermi_surface()` computes the spectral weight on a (k_x, k_y) mesh at a single energy (by default the Fermi level, `e=0.0`), i.e. a single constant-energy cut

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(kx,ky,fs) = h.get_fermi_surface(e=0.0,nk=50,delta=1e-1)
```

Optional arguments - e: energy of the cut - nk: number of k-points per direction - delta: broadening - operator: project/weight the Fermi surface by an operator, giving e.g. a spin- or valley-textured Fermi surface

```
from pyqula.specialhamiltonian import NbSe2
h = NbSe2(soc=0.9) # multi-orbital spin-orbit-coupled Hamiltonian
(kx,ky,fs) = h.get_fermi_surface(e=0.,nk=100,delta=3e-1,operator="sz")
```

`h.get_multi_fermi_surface()` computes the same kind of map at many energies at once, writing one file per energy to a `MULTIFERMISURFACE/` folder – convenient for scanning how the Fermi surface evolves away from the Fermi level

```
import numpy as np
h.get_multi_fermi_surface(energies=np.linspace(-4,4,100),delta=1e-1)
```

Passing `operator="unfold"` together with `nsuper` unfolds the Fermi surface of a defective/disordered supercell back onto the primitive Brillouin zone (see the

"Electronic structure folding and unfolding" section); as with QPI unfolding, the supercell must be built with `store_primal=True`

```
from pyqula import geometry
g0 = geometry.triangular_lattice()
n = 3 # size of the supercell
g = g0.get_supercell(n,store_primal=True)
h = g.get_hamiltonian()
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a point defect

out = h.get_multi_fermi_surface(nk=50,energies=np.linspace(-4,4,100),delta=0.1,
        nsuper=n,operator="unfold")
```

See `examples/readme_examples/fermi_surface/main.py`, `examples/2d/operator_fermi_surface/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions.

Quasiparticle interference

Quasiparticle interference (QPI) maps the momentum-space scattering pattern that a defect or impurity produces, and is what an STM quasiparticle-interference measurement probes. `h.get_qpi()` is only available for 2D Hamiltonians; unlike the other observables here it does not return arrays – it writes its output to disk, one file per energy in an output folder (default `MULTIQPI/`) plus a combined `DOS.OUT`

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice()
h = g.get_hamiltonian(has_spin=False)
h.get_qpi(mode="pm",nk=50,delta=1e-1,energies=np.linspace(-6.,6.,100))
```

Optional arguments - energies: array of energies to compute - nk: number of k-points per direction - delta: broadening - mode: `"pm"` ("poor man's") autoconvolves the actual k-resolved spectral weight of the (possibly defective) system in q-space – the physically meaningful QPI signal for a real scatterer; `"response"` (default) instead computes a cheaper Lindhard-like joint-DOS convolution from the clean band structure only, ignoring wavefunction form factors - nunfold: for a defect embedded in an `nunfoldxnunfold` supercell, unfold the QPI signal back onto the primitive Brillouin zone

A single point defect embedded in a supercell, with the resulting QPI unfolded back onto the primitive cell, is a realistic use case. The supercell must be built with `store_primal=True` so pyqula remembers the primitive-cell reference needed to unfold; `operator="unfold"` then resolves to the corresponding unfolding operator

```
import numpy as np
from pyqula import geometry
```

```
g0 = geometry.honeycomb_lattice()
ns = 2
g = g0.get_supercell(ns,store_primal=True)
h = g.get_hamiltonian(has_spin=False)
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a strong point de

h.get_qpi(mode="pm",delta=1e-2,operator="unfold",nsuper=2,nk=140,nunfold=ns)
```

This is the most expensive snippet in the guide: `mode="pm"` diagonalizes on an `nkxnk` mesh and then autoconvolves the result, so the cost grows quadratically with `nk` and the `nk=140` above takes minutes. Drop to `nk=60` (about 40 seconds) while setting a calculation up, and raise `nk` only for the final figure – the q-space resolution of the QPI pattern is what it buys.

See `examples/2d/multiqpi/main.py` (clean system, `mode="pm"`) and `examples/2d/multiqpi_unfold/main.py` (defect in a supercell, unfolded) for runnable versions.

Real-space-impurity QPI

`h.get_qpi()`'s modes are all reciprocal-space methods (they convolve or scatter k-resolved spectral weight, never touch real-space impurities). `h.get_qpi_impurity()` instead takes the direct route: it builds a supercell of `h`, adds one or more actual real-space impurities to it, computes the real-space LDOS map with ARPACK partial diagonalization (only the eigenstates nearest each requested energy, so this stays tractable for large supercells, unlike full diagonalization), and Fourier transforms that map directly (a discrete sum over the atoms' actual positions, not a grid FFT) to get the QPI(q) signal. Unlike `get_qpi()`, it returns arrays rather than only writing to disk

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
r,ldos_r,q,qpi_q = h.get_qpi_impurity(nsuper=10,
        impurities=[{"position": [0.,0.,0.], "onsite": 3.0}],
        energies=0.3,num_waves=60,nk=2,delta=0.2)
```

Optional arguments - nsuper: supercell size (scalar or `(n1,n2)`) - impurities: list of dicts, each an onsite potential (`{"position": [x,y,z], "onsite": v}`, or `{"index": i, "onsite": v}` for a specific supercell site index) or a vacancy (`{"position": [x,y,z], "vacancy": True}`). A vacancy is modeled as a strong onsite potential rather than true site removal, since deleting sites from a large sparse supercell Hamiltonian would require densifying it - energies: a single energy or an array - num_waves: a starting guess for the number of ARPACK eigenstates nearest the requested energies – automatically grown (more ARPACK calls, not a correctness risk) until the diagonalization covers `margin*delta` past every requested energy and never cuts a degenerate manifold in half, since summing over a partial manifold isn't basis-independent and would

otherwise leak spurious QPI weight, dependent on ARPACK's starting vector, even for a clean (impurity-free) supercell - nk, delta: as in `get_ldos`

The Hamiltonian is kept sparse throughout (the primitive cell is turned sparse before the supercell is built, and impurities are added as a sparse diagonal), so no dense matrix of the supercell's size is ever built. No unfolding step is needed either: this never diagonalizes supercell bands and projects them onto primitive Bloch states (which is what `get_qpi`'s `nunfold`/`store_primal` are for) – it only Fourier transforms a real-space scalar density, evaluated directly at q spanning the full primitive Brillouin zone. `q` is fixed at exactly the `nsuper1`x`nsuper2` points commensurate with the supercell (`nsuper` sets the achievable q *resolution*, not the BZ range); evaluating the direct-sum Fourier transform at any other q would show finite-size leakage even for a perfectly clean system, since only the commensurate points are free of it.

See `examples/2d/qpi_realspace_impurity/main.py` for a runnable version that plots both the real-space LDOS and QPI(q).

Operators

Both when computing band structures, density of states and expectation values we could define operators to filter the results. In this section we elaborate on some important operators that are available, and we comment on their physical meaning.

Operators in pyqula have some important properties. First, for periodic Hamiltonian they can have an intrinsic momentum dependence. Second, pyqula allows for native algebra between them, namely they can be summed or multiplied, automatically accounting for intrinsic momentum depences. Third, they can be non-linear, providing a generalization of matrix operators.

Spin operators

The simplest operators are the spin operators

$$S_\alpha = \sum_n \sigma_\alpha^{\mu\nu} c_{n,\mu}^\dagger c_{n,\nu}$$

with σ_α the Pauli matrices, that can be obtained as

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
sx = h.get_operator("sx") # Spin x component
sy = h.get_operator("sy") # Spin y component
sz = h.get_operator("sz") # Spin z component
```

Location operator

To understand the spatial location of the states we can use the spatial operators, that denote where wavefucntions are located in real space

$$R_\alpha = \sum_{r,s} r_\alpha c^\dagger_{r,s} c_{r,s}$$

with r_α is the component of the position of site r

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
x = h.get_operator("xposition") # x component
y = h.get_operator("yposition") # y component
z = h.get_operator("zposition") # z component
```

Bulk-edge operator

In order to know if a state is located at the edge or in the bulk of the system you can use the bulk-edge location operators. The edge operator takes value 1 for sites on the edge, and 0 for sites in the bulk.

$$hatE = \sum_{r \in \text{Edge},s} c^\dagger_{r,s} c_{r,s}$$

The bulk operator takes value 1 for sites on the bulk, and 0 for sites on the edge.

$$hatB = \sum_{r \in \text{Bulk},s} c^\dagger_{r,s} c_{r,s}$$

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
b = h.get_operator("bulk") # bulk operator
e = h.get_operator("edge") # edge operator
```

Valley operator

For hoenycomb like systems, including aligned and twisted multilayers, an operator that allows to extract the valley degree of freedom can be extracted. This operator takes the form

$$V = i \sum_{\langle\langle ij \rangle\rangle,s} \nu_{ij} \sigma_{ij} c^\dagger_{r_i,s} c_{r_j,s}$$

where $\nu = \pm 1$ and $\sigma = \pm 1$ for clockwise/anticlockwise, sublattice A/B. This the so-called anti-Haldane hopping, and takes opposite values in opposite valleys. It can can be obtained for honeycomb systems as

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
vall = h.get_operator("valley") # valley operator
```

In-plane valley operators

The operator above is the out-of-plane valley pseudospin τ_z. The two remaining components of the valley pseudospin, τ_x and τ_y, can also be obtained, giving access to the full valley vector (τ_x, τ_y, τ_z) – the valley-space analogue of (S_x, S_y, S_z) for real spin. They are built from a chiral Kekule coupling (symmetrized over the 3 inequivalent Kekule registries so the result is exactly C_3-covariant about every atom, not only about special high-symmetry points) rather than from the second-neighbor coupling behind τ_z

```
from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
h = g.get_hamiltonian(has_spin=False)  # get the Hamiltonian
taux = h.get_operator("valley_x") # tau_x
tauy = h.get_operator("valley_y") # tau_y
```

Both require a honeycomb-like geometry with a sublattice index; for a periodic (non-0d) Hamiltonian they additionally require a Kekule-commensurate cell (already a 3x3, or other multiple-of-3, supercell of the primitive honeycomb cell) to be well-defined – a finite (0d) flake needs no such commensurability.

A single vacancy in an otherwise pristine honeycomb flake is an atomically-sharp, intervalley-scattering defect, and induces a vortex in the in-plane valley pseudospin around it – a nice way to see τ_x, τ_y in action

```
from pyqula import islands
from pyqula import spectrum
g = islands.get_geometry(name="honeycomb",n=8,nedges=6)
gv = g.remove(g.get_central()[0]) # flake with a single vacancy
hv = gv.get_hamiltonian(has_spin=False)
dvx = spectrum.real_space_vev(hv,operator=hv.get_operator("valley_x"))
dvy = spectrum.real_space_vev(hv,operator=hv.get_operator("valley_y"))
```

`h.add_valley_exchange(v)`, with `v=(vx,vy,vz)`, adds a valley-space exchange term $\vec{v} \cdot (\tau_x, \tau_y, \tau_z)$ to the Hamiltonian – the valley-pseudospin analogue of `add_exchange` for real spin

```
from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
```

```
h = g.get_hamiltonian(has_spin=False)
h.add_valley_exchange([0.1,0.05,0.2]) # (vx,vy,vz)
```

See `examples/0d/valley_vortex_vacancy/main.py` and `examples/2d/valley_vortex/main.py` for runnable versions of the vacancy-vortex example.

Nambu operators

In the presence of superconductivity, you can project onto the electron or hole component of the Nambu spinor using the electron-hole operators. The Hamiltonian must already be in the Nambu (BdG) basis – i.e. some pairing has been added – otherwise there is no hole sector to project onto and these raise

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
h.add_swave(0.2) # add pairing, doubling the basis into Nambu space
electron = h.get_operator("electron") # electron component
hole = h.get_operator("hole") # hole component
(k,e,c) = h.get_bands(operator=electron) # electron weight of each state
```

Berry curvature operator

The Berry curvature operator is a first example of an operator that is intrinsically momentum dependent. The Berry curvature operator is defined as

$$O|\Psi_k\rangle = \Omega(k,\epsilon_k)|\Psi_k\rangle$$

where $\Omega(k,\omega)$ is the Berry curvature evaluated at the momentum k and energy ω of the eigenstate $|\Psi\rangle$. In particular, this operator allows to directly see the contribution to the Berry curvature of different states in the band structure.

Inverse participation ratio operator

So far we have considered operators that are linear, namely that fufill the condition

$$A(|\Psi_1\rangle + |\Psi_2\rangle) = A|\Psi_1\rangle + A|\Psi_2\rangle$$

There is however one operator that it is interesting to consider that does not fufill such condition. The operator is the so-called inverse participation ration, which we define as

$$O|\Psi\rangle = \sum_i |\langle i|\Psi\rangle|^4 |\Psi\rangle$$

In particular, the previous operator allows to identify states that are highly localized in a few lattice sites, becoming useful to highlight impurity states and localized modes.

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian()  # get the Hamiltonian
h.add_onsite(0.3) # add a sublattice imbalance
ipr = h.get_operator("IPR") # IPR operator
```

Superconductivity

Up to now we have focused on Hamiltonians that contain only normal terms, namely that the full Hamiltonian can be written as

$$H_0 = \sum_{ijss'} t_{ijss'} c^\dagger_{i,s} c_{j,s'}$$

where ij runs onver sites and ss' over spins.

In the presence of superconductivity, an anomalous term appears in the Hamiltonian taking the form

$$H_{SC} = \sum_{ijss'} \Delta^{ss'}_{ij} c_{i,s} c_{j,s'} + h.c.$$

To solve the Hamiltonian

$$H = H_0 + H_{SC}$$

we define a Nambu spinor that takes the form

$$\Psi_n = \begin{pmatrix} c_{n,\uparrow} \\ c_{n,\downarrow} \\ c^\dagger_{n,\downarrow} \\ -c^\dagger_{n,\uparrow} \end{pmatrix}$$

and rewrite the Hamiltonian as

$$H = \Psi^\dagger \mathcal{H} \Psi$$

where $\mathcal{H}$ is the nambu Hamiltonian. In this new basis, the Hamiltonian can be written in a diagonal form as

$$H = \sum_{\alpha} \epsilon_{\alpha} \Psi_{\alpha}^{\dagger} \Psi_{\alpha}$$

where ϵ_{α} are the Nambu eigenvalues.

s-wave superconductivity

The simplest form of superconductivity is spin-singlet s-wave superconductivity. A minimal superconducting term of this form can be written as

$$H_{SC} = \Delta_0 \sum_{n} c_{n,\uparrow} c_{n,\downarrow} + h.c.$$

In the following, we address the electronic structure of a triangular lattice with s-wave superconductivity, whose Hamiltonian takes the form

$$H = H_0 + H_{SC}$$

with

$$H_0 = \sum_{\langle ij \rangle} c_i^{\dagger} c_j + h.c.$$

The previous Hamiltonian can be computed for $\Delta_0 = 0.2$ as

```
from pyqula import geometry
g = geometry.triangular_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_swave(0.2) # add s-wave superconductivity
(k,e) = h.get_bands() # compute band structure
```

Note that due to the BdG nature of the Hamiltonian, the bandstructure shows both the electron and hole states

Interactions at the mean-field level

In this section we address how interactions can be treated at the mean-field level.

The collinear Hubbard model

We will start with the simplest interaction term, a local repulsive interaction in a spinful system. Our full Hamiltonian takes the form

$$H = \sum_{\langle ij \rangle} c_i^{\dagger} c_j + h.c. + U \sum_{i} c_{i,\uparrow}^{\dagger} c_{i,\uparrow} c_{i,\downarrow}^{\dagger} c_{i,\downarrow}$$

The interaction term $U\sum_i c^\dagger_{i,\uparrow}c_{i,\uparrow}c^\dagger_{i,\downarrow}c_{i,\downarrow}$ is solved at the mean-field level. The mean-field approximation consists on replacing the previous four fermion operator, by all the terms that arise by taking expectation value in two of the fermions. In particular, in its simplest collinear form, the mean-field term takes the form

$$H_U^{MF} = U\sum_i \langle c^\dagger_{i,\uparrow}c_{i,\uparrow}\rangle c^\dagger_{i,\downarrow}c_{i,\downarrow} + c^\dagger_{i,\uparrow}c_{i,\uparrow}\langle c^\dagger_{i,\downarrow}c_{i,\downarrow}\rangle$$

where $\langle\rangle$ denotes the ground state expectation value of those operators. The full Hamiltonian thus takes the form

$$H^{MF} = \sum_{\langle ij\rangle} c^\dagger_i c_j + h.c. U\sum_i \langle c^\dagger_{i,\uparrow}c_{i,\uparrow}\rangle c^\dagger_{i,\downarrow}c_{i,\downarrow} + c^\dagger_{i,\uparrow}c_{i,\uparrow}\langle c^\dagger_{i,\downarrow}c_{i,\downarrow}\rangle$$

As a result, the mean-field Hamiltonian depends on the specific ground state of the system, and the ground state depends of course on the specific mean-field Hamiltonian. The previous circular dependence between the ground state and the mean-field Hamiltonian gives rise to a selfconsistent problem.

This selfconsistent condition is solved as follows. We start with an initial guess for the full many-body ground state, that we call $|GS_0\rangle$. With this initial state, we compute the mean-field Hamiltonian H_0^{MF}. This mean-field Hamiltonian allows to compute a new many-body ground state $|GS_1\rangle$, which in turn allows to compute a new mean-field Hamiltonian H_1^{MF}. The previous algorithm is represented as

$$|GS_0\rangle \rightarrow H_1^{MF} \rightarrow |GS_1\rangle \rightarrow H_1^{MF} \rightarrow |GS_2\rangle \rightarrow H_2^{MF} \rightarrow ...$$

This iterative calculation is performed until $H_n^{MF} = H_{n+1}^{MF}$, at which point the algorithm has converged.

Two important notes can we taken from the previous approach. First, the final solution may be sensitive to the initial guess for the ground state. This guess corresponds to the initialization of the Hamiltonian, and it can be important for system whose eenrgy landscape has several local minima. A second point is that the update procedure from one iteration to the next can be done adiabatically, or very suddenly. This corresponds to the mixing between solutions, and for systems close to the critical point can lead to tricky convergence.

Let now show an example of a mean-field calculation. We will take now a square lattice, make a 2x2 supercell and include local repulsive interactions at half filling. The obtained ground state is an antiferromagnetic Neel state that opens a gap at half filling

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
g = g.get_supercell([2,2]) # generate a 2x2 supercell
```

```
h = g.get_hamiltonian() # create hamiltonian of the system
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                 mf="random") # perform SCF
(k,e) = h.get_bands() # calculate band structure
m = h.get_magnetization() # get the magnetization
```

Non-collinear Hubbard model

In the mean-field ansatz considered above, only a single term in the Wick contraction was considered. This term is the collinear term in the z-direction, and allows accounting for solutions that have magnetization in the z-direction. However, in the presence of frustration, external magnetic field or spin-orbit coupling, the magnetization of a system may be non-collinear and pointing in an arbitrary direction. To account for that phenomenology, the mean-field Hamiltonian must include the non-collinear term that takes the form

$$H_U^{ncMF} = -U \sum_i \langle c^\dagger_{i,\downarrow} c_{i,\uparrow} \rangle c^\dagger_{i,\uparrow} c_{i,\downarrow} + h.c.$$

When including this additional term, the full mean-field Hubbard Hamiltonian is rotationally invariant, meaning that it respects SO(3) spin rotational symmetry. This rotationally symmetric form is the default form implemented in the library.

With the previous point in mind, we now solve a system that develops a non-collinear magnetic state. We take the square lattice considered in the section above, and we add an external magnetic field. The competition between Zeeman energy and antiferromagnetic correlations gives rise to a canted magnetic state

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
g = g.get_supercell([2,2]) # generate a 2x2 supercell
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_zeeman([0.,0.,0.1]) # add out-of-plane Zeeman field
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                 mf="random") # perform SCF
(k,e,c) = h.get_bands(operator="sz") # calculate band structure
m = h.get_magnetization() # get the magnetization
```

Superconducting mean-field

In the cases above we focused on local repulsive interactions that promote collinear or non-collinear magnetism. However, local interactions can promote a different type of symmetry breaking, in particular gauge symmetry breaking associated to superconductivity. The emergence of superconductivity is associated to one of the Wick contractions of the mean-field, the anomalous term, that takes the form

$$H_U^{aMF} = U \sum_i \langle c_{i,\uparrow} c_{i,\downarrow} \rangle c_{i,\downarrow}^\dagger c_{i,\uparrow}^\dagger + h.c.$$

The previous term in the mean-field Hamiltonian can become non-zero for $U < 0$, and yields an interaction induced superconducting state. This term in the mean-field Hamiltonian is automatically accounted for in Hamiltonian with Nambu degree of freedom, of course apart from the collinear and non-collinear terms in the mean-field. We show below how an interaction induced superconducting state can be computed with pyqula

```
from pyqula import geometry
import numpy as np
g = geometry.triangular_lattice() # geometry of a triangular lattice
h = g.get_hamiltonian()  # get the Hamiltonian
h.setup_nambu_spinor() # setup the Nambu form of the Hamiltonian
h = h.get_mean_field_hamiltonian(U=-1.0,filling=
                  0.15,mf="swave") # perform SCF
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                  energies=np.linspace(-1.0,1.0,100))
```

Long range interactions

Up to now we have considered interacting Hamiltonians that only have local (attractive or repulsive) Hubbard interactions. In the following we are going to consider systems that have many-body interactions also to a certain number of neighbors. Long range interactions are crucial to stabilize specific symmetry broken states, and in particular charge density waves, Peierls instabilities and unconventional superconductivity. The full Hamiltonian we will consider takes the form

$$H = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow} + V_1 \sum_{\langle ij \rangle, s, s'} c_{i,s}^\dagger c_{i,s} c_{j,s'}^\dagger c_{j,s'}$$

where U parametrizes onsite interactions and V_1 interactions between first neighbors. The previous Hamiltonian gives rise to a variety of terms when performing a mean-field decoupling. By default, pyqula includes all the Wick contractions of the mean-field, and in the presence of Nambu spinors it includes all the anomalous contractions. Let us now briefly elaborate on some of the additional terms that arise due to the first neighbor interaction V_1.

The first term is the charge order term, that takes the form

$$H^{MF} \sim \langle c_{i,s}^\dagger c_{i,s} \rangle c_{j,s'}^\dagger c_{j,s'}$$

this term can give rise to a different charge imbalance between different sites, and it leads to charge density wave states.

The second term we consider is the bond order, that takes the form

$$H^{MF} \sim \langle c^\dagger_{i,s} c_{j,s} \rangle c^\dagger_{j,s} c_{i,s} + h.c.$$

which leads to an interaction-enhanced hopping. If this happens in a non-uniform way in the system, the resulting state has a Peierls distortion.

Among the anomalous terms, the mean-field Hamiltonian can generate

$$H^{MF} \sim \langle c^\dagger_{i,\uparrow} c^\dagger_{j,\uparrow} \rangle c_{j,\downarrow} c_{i,\downarrow} + h.c.$$

$$H^{MF} \sim \langle c^\dagger_{i,\downarrow} c^\dagger_{j,\downarrow} \rangle c_{j,\uparrow} c_{i,\uparrow} + h.c.$$

$$H^{MF} \sim \langle c^\dagger_{i,\uparrow} c^\dagger_{j,\downarrow} \rangle c_{j,\downarrow} c_{i,\uparrow} + h.c.$$

where the first two-terms corresponds to the odd superconducting order, and the third term account both for even and odd orders.

Below, we show an example in which an interaction-induced spin-triplet term is generated. By considering a electronic structure with a large Zeeman splitting and attractive first neighbor interactions, a state with non-zero $\Delta_{\uparrow\uparrow}$ and $\Delta_{\downarrow\downarrow}$ emerges.

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice() # generate the geometry
h = g.get_hamiltonian() # create Hamiltonian of the system
h.add_exchange([0.,0.,1.]) # add exchange field
h.setup_nambu_spinor() # initialize the Nambu basis
# perform a superconducting non-collinear mean-field calculation
h = h.get_mean_field_hamiltonian(V1=-1.0,
                     filling=0.3,mf="random")
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                   energies=np.linspace(-2.0,2.0,400))
```

Spin-spin exchange interactions

The interactions considered above are all of density-density type, $c^\dagger_{i,s} c_{i,s} c^\dagger_{j,s'} c_{j,s'}$. A different kind of interaction, relevant for localized-moment magnetism, is a direct spin-spin coupling $\vec{S}_i \cdot \vec{S}_j$ between the (Pauli) spin operators at two sites,

$$H = J_z \sum_{\langle ij \rangle} S_i^z S_j^z + J_x \sum_{\langle ij \rangle} S_i^x S_j^x + J_y \sum_{\langle ij \rangle} S_i^y S_j^y$$

with $J > 0$ the antiferromagnetic (Heisenberg) sign convention and $J < 0$ favoring a ferromagnetic instability. Writing $S_i^z = (n_{i,\uparrow} - n_{i,\downarrow})/2$,

$$S_i^z S_j^z = \tfrac{1}{4} \left(n_{i,\uparrow} n_{j,\uparrow} - n_{i,\uparrow} n_{j,\downarrow} - n_{i,\downarrow} n_{j,\uparrow} + n_{i,\downarrow} n_{j,\downarrow} \right)$$

is already a density-density interaction between spin-orbitals, so $S_i^z S_j^z$ is solved with exactly the same Hartree-Fock machinery as the $U/V_1/V_2/V_3$ interactions above – `h.get_szsz_mean_field_hamiltonian(J1=...)` (first-neighbor J_z; `J2`/`J3` add second/third neighbors, `Jr` a general distance-dependent coupling, following the same convention as `V1`/`V2`/`V3`/`Vr` in `get_mean_field_hamiltonian`). $S_i^x S_j^x$ and $S_i^y S_j^y$ are obtained by a global spin rotation that maps the x (or y) axis onto the computational z axis, solving the $S^z S^z$ problem there, and rotating the converged Hamiltonian back – `h.get_sxsx_mean_field_hamiltonian(...)` and `h.get_sysy_mean_field_hamiltonian(...)`. Because the bare interaction is SU(2)-symmetric, the converged total energy of `SzSz`, `SxSx` and `SySy` at the same coupling only differs by which axis the moment orders along.

```
from pyqula import geometry
g = geometry.chain() # a chain, prone to ferromagnetic order away from half filling
h = g.get_hamiltonian(has_spin=True)
h = h.get_szsz_mean_field_hamiltonian(J1=-2.0,filling=0.2,
                                      mf="ferroZ", # ferromagnetic Sz-Sz coupling
                                      nk=10,mix=0.3,maxite=300)
m = h.get_magnetization() # uniform moment along z
```

Getting these SCF loops to converge. All the mean-field entry points return `None` instead of a Hamiltonian when the loop does not converge, so check the result before using it. Two defaults are worth overriding explicitly for a partially filled metal like this chain:

- `maxite=None` (the default) means *no iteration limit*, so a loop that settles into a limit cycle instead of a fixed point never returns. Always pass a finite `maxite` while exploring parameters – you then get a `None` and a "no convergence" message in a few seconds instead of a hung session
- `nk=8` (the default k-mesh) is often the actual culprit rather than the mixing. At `filling=0.2` this chain does *not* converge at `nk=8` for any mixing, because the mesh does not resolve the Fermi points and the occupied set flips between iterations; `nk=10` converges in a fraction of a second. If an SCF refuses to converge, change `nk` before reaching for a smaller `mix`

`mix` (0.1 by default, linear mixing of successive mean fields) is the knob for a loop that oscillates around a fixed point rather than one that never approaches one;

`maxerror` (1e-5) sets the convergence threshold. `tests/scf/` is a good source of known-converging parameter sets for each coupling.

The three channels can also be combined into a single anisotropic-exchange SCF loop, `h.get_exchange_mean_field_hamiltonian(Jx1=...,Jy1=...,Jz1=...)`, which decouples the z channel directly and the x/y channels through the same rotate-solve-rotate-back trick, each SCF iteration:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_exchange_mean_field_hamiltonian(Jz1=-1.0,Jx1=-0.5,
                                          filling=0.2,mf="ferroZ",
                                          nk=10,mix=0.3,maxite=300)
```

Density-density interactions ($U/V_1/V_2/V_3/V_r$, as in `get_mean_field_hamiltonian`) and spin-spin exchange can also be solved together, self-consistently, in a single combined SCF loop with `h.get_combined_mean_field_hamiltonian(U=...,V1=...,J1=...,...)`. This is not new physics: a density-density interaction and $S_i^z S_j^z$ are both density-density interactions in the spin-orbital basis (just with a different sign pattern across the four spin blocks), and the Hartree-Fock decoupling is linear in the interaction, so the density-density contribution is simply added into the same z-channel matrix the exchange term already uses. Exchange here follows a $V_1/V_2/V_3$-like convention: $J_1/J_2/J_3$ ($+J_r$) are isotropic Heisenberg couplings, $J(S_i^x S_j^x + S_i^y S_j^y + S_i^z S_j^z)$, for the first/second/third neighbor shells, and $J_{1x}/J_{1y}/J_{1z}$ are an optional anisotropic correction added on top of J_1 for the first-neighbor shell only (e.g. the effective first-neighbor J_z coupling is $J_1 + J_{1z}$); all default to 0

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,
                                          filling=0.2,mf="ferroZ")
```

The same combined SCF loop can instead get its density matrix through a per-k Chebyshev-moment (Kernel Polynomial Method) expansion, never diagonalizing the Bloch Hamiltonian, with `integration="kpm"`:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                          mf="ferroZ",integration="kpm")
```

This is meant for large/sparse systems where per-iteration exact diagonalization is the bottleneck – **but currently measures far slower than `integration="ed"` at the small/moderate sizes actually tested (order 100-500 sites, see the reference entry below for numbers)**, so benchmark before relying on it for a given system.

The same combined SCF loop can instead be solved with a JAX-derivative-based solver, `use_jax=True`: rather than the default plain-mixing loop, this builds a JAX-differentiable version of one SCF iteration $x = f(x)$ and drives it to its fixed point with a genuine root-finder using JAX-computed derivatives

of f (`solver="newton"`, the default once `use_jax=True`), a matrix-free variant that scales to larger systems (`solver="newton_krylov"`), by minimizing the squared residual $\|f(x) - x\|^2$ as a proper nonlinear least-squares problem via matrix-free Levenberg-Marquardt (`jax.jvp`/ `jax.vjp` plus `scipy`'s `lsqr`, `solver="error_gradient"`), or with a robust black-box mixing scheme, `solver="broyden_mixing"` – a regularized, limited-memory multisecant form of Broyden's second method following Marks & Luke, *Robust Mixing for Ab-Initio Quantum Mechanical Calculations* (arXiv:0801.3098). Unlike the root-finder/gradient-based solvers above, it only ever evaluates f itself (no Jacobian, no autodiff), tracking the last few SCF steps as simultaneous secant conditions, regularizing the resulting least-squares solve (Tikhonov), and adaptively bounding the step length – this is the combination the paper credits for converging on cases (e.g. "charge sloshing" between two badly-scaled subsets of the mean field) that defeat a single fixed linear-mixing factor. It first runs a plain-linear- mixing warm-up (reusing `mix`) until the residual drops below a threshold (`warmup_tol`, default 1e-2) before switching on the multisecant machinery – benchmarked across several small (5-13 atom) systems, starting the multisecant phase directly on a cold guess (the paper's own literal algorithm) regularly failed to converge, while the warm-up fixed every observed case and also converged 2-10x faster than plain linear mixing alone; see `scftk.broydenmixing`'s module docstring for the algorithm and that benchmark. See the reference entry below for the full solver list and scope restrictions (normal-state only, no `constrains`), and `scftk.vjinteraction_jax`'s module docstring for why `solver="error_gradient"` minimizes the SCF residual rather than the physical free energy directly.

Which solver is most likely to converge at all (as opposed to fastest) matters more than raw speed when the system's symmetry properties aren't known in advance (e.g. no explicit symmetry-breaking bias applied to the Hamiltonian). Measured across a range of system sizes and both biased and fully generic (unbiased) Hamiltonians, `solver="error_gradient"` was consistently the most robust of the four `use_jax=True` solvers, converging in nearly every case tried – including the hardest one, a larger unbiased system where `"linear_mixing"` failed outright and `"broyden_mixing"` converged only a minority of the time (outside the small-system regime it was validated on above). `"newton_krylov"` was also reliable at larger sizes but considerably slower there, and was the least robust solver of the four on small systems, where its GMRES step can fail outright against a near-singular Jacobian. Net recommendation: default to `solver="error_gradient"` for a generic system whose symmetry isn't known to be already broken; reach for `"newton_krylov"`/`"broyden_mixing"` instead once the system is known to be well-conditioned (or explicitly biased) and the extra speed matters more than robustness.

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                          mf="ferroZ",use_jax=True,
                                          solver="newton")
```

Needs the optional `jax` extra (`pip install pyqula[jax]`).

All of the spin-spin exchange functions above also work on BdG (Nambu) Hamiltonians (`h.turn_nambu()`/`h.setup_nambu_spinor()`). `get_szsz_mean_field_hamiltonian`/`get_sxsx_mean_f` need no special handling: `get_mean_field_hamiltonian`'s existing Hartree-Fock-plus-anomalous decoupling already dispatches generically for any density-density-shaped interaction, including $S_i^z S_j^z$'s. `get_combined_mean_field_hamiltonian`/`get_exchange_mean_f` decouple the exchange (J) channels with the same full normal-plus-anomalous treatment as $U/V_1/V_2/V_3$ for a Nambu Hamiltonian, so exchange can itself induce superconducting pairing, not just density-density interactions: an antiferromagnetic isotropic J alone (no U/V at all), seeded with a small coherent pairing guess (e.g. `h.add_swave(0.1)` on top of the Hamiltonian used as `mf`; a purely random guess has no reliable overlap with this instability and often relaxes back to zero pairing instead), can spontaneously decouple into a purely superconducting, singlet-paired state (the same RVB-like mechanism behind exchange-driven superconductivity), while the ferromagnetic sign has no such pairing tendency and stays magnetic. A state with both magnetic and superconducting order can also still emerge from combining an exchange field with an attractive V_1. CAVEAT: the reported `total_energy` only ever subtracts the normal (Hartree-Fock) double-counting correction, never a matching one for the anomalous/pairing channel, so it is systematically off whenever any channel (exchange or V/U) converges to a nonzero pairing amplitude; the converged Hamiltonian itself is unaffected by this, only the `total_energy` scalar:

```
h = g.get_hamiltonian(has_spin=True)
h.add_exchange([0.,0.,0.3])
h.turn_nambu()
h = h.get_combined_mean_field_hamiltonian(V1=-1.0,J1z=-0.3,
                                          filling=0.3,mf="random")
```

Abrikosov-pseudofermion (spinon) mean field for Heisenberg models

Rather than seeding a spin-spin exchange on top of an existing tight-binding Hamiltonian, a pure spin-$\frac{1}{2}$ Heisenberg model $H = J\sum_{\langle ij\rangle} \vec{S}_i \cdot \vec{S}_j$ can be treated on its own terms with the Abrikosov-pseudofermion (parton) representation $\vec{S}_i = \frac{1}{2} f_i^\dagger \vec{\sigma} f_i$ (Savary & Balents, *Quantum Spin Liquids: a review*, arXiv:1601.03742, Sec. 4): each spin is written in terms of an auxiliary ("spinon") fermion subject to the hard local constraint $f_i^\dagger f_i = 1$, exactly one fermion per site, and the exchange term is Wick-decoupled into an RVB bond order parameter $\chi_{ij} = \langle f_i^\dagger f_j \rangle$ – physically the same Fock/ Hartree-Fock decoupling `get_combined_mean_field_hamiltonian`'s J channel already performs, just on a Hamiltonian with zero bare hopping (a pure spin model has no bare electron kinetic term) and with the local constraint enforced at *every* site individually, not only on lattice average. `SpinonHamiltonian` (`pyqula.spinon`) packages exactly this:

```
from pyqula import geometry
from pyqula.spinon import SpinonHamiltonian

g = geometry.triangular_lattice() # a canonical frustrated-Heisenberg lattice
h = SpinonHamiltonian(g) # zero bare hopping -- couplings come from J1/J2/...
h2 = h.get_mean_field_hamiltonian(J1=1.0,nk=12,mix=0.1,maxerror=1e-4)

h2.local_occupation   # <n_i> per site -- exactly 1.0 at convergence
h2.constraint_lambda  # converged per-site Lagrange multiplier (local chemical potential)
h2.get_bands()        # spinon dispersion
```

`filling=` cannot be passed to `SpinonHamiltonian.get_mean_field_hamiltonian` – the representation is only valid at exactly one fermion per site, so it is always requested internally as the per-site array `get_combined_mean_field_hamiltonian`'s own `filling` kwarg now accepts (one target per site, instead of only a single lattice-averaged Fermi level), enforced via a per-site Lagrange multiplier warm-started and co-converged with the RVB mean field across the same SCF loop; `scf.converged` (equivalently, a non-`None` return here) already implies the local constraint converged to within `maxerror`, not only the mean field itself. Only the U(1) (RVB bond-only) ansatz is implemented – a Z2 ansatz (allowing the pairing/anomalous channel J can also induce, as above) would need a Nambu-doubled `SpinonHamiltonian`, not yet supported. All other `get_mean_field_hamiltonian` kwargs (`mf`, `nk`, `mix`, `maxerror`, `maxite`, `constrains`, an additional `V1`/`V2`/`V3`/`U` density-density term, ...) are forwarded unchanged.

On a frustrated lattice (triangular, kagome, ...) the converged state is ansatz-dependent, not unique: several distinct self-consistent RVB flux sectors can coexist at the same J, and which one an unseeded random `mf` guess lands on is itself part of the physics, not SCF noise – "it is not possible to search for all possible self-consistent mean field solutions... calculations are usually carried out by assuming a particular decoupling scheme" (Savary & Balents, Sec. 4.1). A 1-site-unit-cell chain has a unique solution (no frustration), so repeated calls agree to within `maxerror`; on a frustrated lattice, pass an explicit `mf=` to select a definite ansatz deliberately rather than comparing energies across differently-seeded runs.

An external Zeeman/magnetic field couples to $\vec{S}_i = \frac{1}{2} f_i^\dagger \vec{\sigma} f_i$ exactly, not via any mean-field decoupling (it is already bilinear in f), so it is added as an ordinary single-particle term with the same `Hamiltonian.add_zeeman`/`add_exchange` used everywhere else in pyqula – call it on the `SpinonHamiltonian` instance *before* `get_mean_field_hamiltonian`:

```
h = SpinonHamiltonian(g)
h.add_zeeman([0., 0., 0.3])            # or h.add_exchange([0.,0.,0.3])
h2 = h.get_mean_field_hamiltonian(J1=1.0, nk=12)
h2.get_magnetization()                 # induced <S> per site
```

`add_zeeman`'s argument is the coefficient of $\vec{\sigma}$ (Pauli matrices), not of $\vec{S} = \vec{\sigma}/2$, so the physical field h in $H = -h \cdot S_i$ is twice the value passed in – the same convention `add_exchange` uses on an ordinary electronic Hamiltonian elsewhere in this guide. The local one-fermion-per-site constraint is a total-occupation constraint, not a spin constraint, so it stays exactly satisfied under a field while $\langle S_i \rangle$ is free to grow with it, saturating once the field dominates J (see `tests/spinon/test_spinon_zeeman.py`).

Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice

The Kondo lattice / periodic Anderson model – localized moments exchange-coupled to a conduction electron at the same site – is the standard minimal model of heavy fermion compounds. Following P. Coleman, *Heavy Fermions: electrons at the edge of magnetism*, arXiv:cond-mat/0612006, Sec. III.C, its Coqblin-Schrieffer form is $H = \sum_k \epsilon_k c_k^\dagger c_k + \frac{J}{N} \sum_j S_{ab}(j) c_{jb}^\dagger c_{ja}$ ($N = 2$ for a spin-$\frac{1}{2}$ moment – **not** the coefficient of a bare $J\vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term, see the caveat below). Each moment is represented by an Abrikosov pseudofermion $\vec{S}_j = \frac{1}{2} f_j^\dagger \vec{\sigma} f_j$ subject to the constraint $f_j^\dagger f_j = 1$, and the exchange term is Hubbard-Stratonovich decoupled into a self-consistent hybridization field $V_j = -\frac{J}{2}\langle f_j^\dagger c_j \rangle$ (a "composite fermion", half electron and half spin-flip) plus a Lagrange multiplier λ_j enforcing the local constraint – physically the large-N ($N = 2$) Read-Newns saddle point of the Kondo-lattice path integral. `KondoLatticeHamiltonian` (`pyqula.kondolattice`) packages this: given a conduction-electron Hamiltonian, it fuses on a second, initially decoupled sublattice of localized f-sites (one per conduction site) with zero bare hopping, and self-consistently solves for V_j and λ_j:

```
from pyqula import geometry
from pyqula.kondolattice import KondoLatticeHamiltonian

gc = geometry.chain()
hc = gc.get_hamiltonian(has_spin=True) # conduction electrons
h = KondoLatticeHamiltonian(hc)

seed = ([0.3+0.0j],[0.0]) # (V,lam) -- see the caveat below for why
h2 = h.get_mean_field_hamiltonian(J=1.5,filling=0.15,nk=200,mf=seed)

h2.local_occupation   # <n_f> per localized site -- exactly 1.0 at convergence
h2.hybridization      # converged V per localized site
h2.constraint_lambda  # converged per-site Lagrange multiplier
```

`J` is Coleman's Coqblin-Schrieffer coupling (entering the interaction as J/N with $N = 2$), not the coefficient of a bare $J\vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term – the two differ by a numerical factor that Coleman's Eq. 73-78 already fixes, so this class follows the paper's convention exactly. `filling` sets a lattice-wide chemical

potential *once*, from the bare ($V = 0$) bands, and holds it fixed through the SCF loop rather than re-solving it every iteration (Coleman's Eq. 83 is a fixed-μ, grand-canonical Hamiltonian; the electron count is meant to float, even expand, as V, λ converge, Eq. 91-92) – the local $\langle n_f \rangle = 1$ constraint is enforced separately by λ_j, not by `filling`. All other `get_mean_field_hamiltonian` kwargs (`mf`, `nk`, `mix`, `maxerror`, `maxite`, `T`) are forwarded to the SCF loop unchanged.

$V = 0$ **is always itself a self-consistent solution**, exactly like the trivial root of the BCS gap equation – an unseeded run (`mf=None`, the default) starts there and stays there even for a `J` that also supports a genuine hybridized state, so a nonzero seed (as above) is generally needed to find it. Where both solutions coexist, the hybridized state is the true (lower-energy) ground state. **Avoid a `filling` that lands the chemical potential inside the bare f-sector's flat, macroscopically degenerate band** (at $V = 0$, every f-orbital sits at exactly λ, so a wide range of `filling` values – roughly 0.25-0.75 for a single conduction orbital per site – all give exactly the same, numerically ill-posed starting point); `filling=0.15` above keeps μ inside the dispersing conduction band instead. **The finite Fermi-Dirac smearing `T` this SCF loop necessarily runs at** (needed for the λ feedback's numerical stability – see `scftk.kondolattice.kondo_lattice_mean_field`'s docstring) turns the textbook, continuous $T_K = D\, e^{-1/(J\rho)}$ onset into a genuine finite-temperature Kondo crossover: below a T-dependent threshold in J, thermal smearing washes out the hybridization entirely and $V = 0$ becomes the *only* self-consistent solution, and right at the threshold V jumps directly to an $O(1)$ value rather than growing continuously from zero.

An external Zeeman/magnetic field couples exactly to both fermion species here (the conduction electron and the localized moment $\vec{S}_j = \frac{1}{2} f_j^\dagger \vec{\sigma} f_j$, already bilinear in f), so – exactly as for `SpinonHamiltonian` above – it is added as an ordinary single-particle term with `add_zeeman`/`add_exchange`, called on the `KondoLatticeHamiltonian` instance *before* `get_mean_field_hamiltonian`:

```
h = KondoLatticeHamiltonian(hc)
h.add_zeeman([0., 0., 0.05])
h2 = h.get_mean_field_hamiltonian(J=1.5, filling=0.15, nk=150, mf=seed)
```

`add_zeeman` applies to every site of the fused geometry, i.e. both the conduction and the f sublattice (offset in z) – pass a position- dependent callable instead of a constant vector to target only one of them. The $\langle n_f \rangle = 1$ constraint (a total-occupation, not spin, constraint) stays exact under a field. A field competes with the Kondo singlet: the self-consistent $|V|$ *shrinks* as the field grows at fixed J, and a strong enough field destroys the hybridized state – genuine physics, not a bug, and the SCF correctly reports non-convergence (`None`) there rather than a spuriously small but nonzero V, exactly the "decays toward the always-self-consistent $V = 0$ branch" signal a subcritical J already produces above (see `tests/kondolattice/test_kondolattice_zeeman.py`).

Spatially resolved density of states

```
from pyqula import islands
g = islands.get_geometry(name="honeycomb",n=3,nedges=3) # get an island
h = g.get_hamiltonian() # get the Hamiltonian
h.get_multildos(projection="atomic") # get the LDOS
```

Electronic structure folding and unfolding

Building a supercell folds the bands of the primitive cell into the smaller supercell Brillouin zone. The inverse operation, unfolding, recovers the primitive-cell-like spectral weight of a supercell calculation (e.g. a defect, a moire pattern, or a non-primitive choice of cell), and is essential to compare supercell calculations directly against ARPES-like band structures. Unfolding is implemented as a special operator, `"unfold"`, that projects onto the Bloch states of the primitive cell; it can be passed to any of the k-resolved observables (`h.get_bands()`, `h.get_kdos_bands()`, `h.get_multi_fermi_surface()`...). It requires the supercell to have been built keeping track of the primitive geometry

```
from pyqula import geometry
import numpy as np
g = geometry.honeycomb_lattice() # primitive geometry
n = 3
gs = g.get_supercell(n,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1) # unfolded spectral function
```

`d` holds the unfolded spectral weight at each `(k,e)`; plotting a scatter of `k,e` colored/sized by `d` recovers the primitive-cell band structure out of the supercell calculation. The same `operator="unfold"` can be passed to `h.get_multi_fermi_surface()` to unfold constant-energy cuts. See `examples/2d/unfolding/main.py`, `examples/1d/unfolding/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions.

Unfolding also works when atoms have been removed from the supercell (e.g. `gs = gs.remove([...])` before `gs.get_hamiltonian()`), such as a vacancy or an irregularly-shaped flake cut out of a supercell: pyqula matches each remaining atom back to its primitive-cell replica by position instead of assuming every replica is fully present, so no extra arguments are needed — `operator="unfold"` transparently falls back to this slower, defect-tolerant path whenever the supercell's atom count doesn't match a complete replication of the primitive cell, and uses the original fast path otherwise. This position match requires the remaining atoms to sit exactly where they were in the original, undefective supercell, so don't call anything that moves atoms (e.g. `gs.center()`, a geometry relaxation) between `gs.remove(...)` and `gs.get_hamiltonian()` — doing so raises a `ValueError` rather than silently unfolding onto the wrong replica.

Unfolding also works for a general, non-diagonal/non-orthogonal supercell, built by passing a 3x3 integer matrix `M` to `get_supercell` instead of a plain `(n1,n2,...)` size (`gs.a1,gs.a2,gs.a3` become integer combinations of the primitive vectors, `gs = M @ g`). No change is needed at the unfolding call site — `get_supercell(M,...)` records, per surviving atom, which primitive replica it came from, and `operator="unfold"` reads that bookkeeping directly (both for a complete supercell and after removing atoms):

```
g = geometry.honeycomb_lattice() # primitive geometry
M = [[2,1,0],[0,1,0],[0,0,1]] # non-diagonal supercell matrix, det(M)=2
gs = g.get_supercell(M,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1) # unfolded spectral function
```

This bookkeeping-based path only supports 1D/2D lattices (matching the diagonal case); a 3x3 `M` on a 3D bulk geometry is not yet implemented.

Surface spectral functions

In this section we address how we can compute the surface spectral function of a semi-infinite system, i.e. a system that is bulk-like far from a boundary but is cleaved along one direction. This is obtained from the surface Green's function, computed with a renormalization (decimation) technique for the semi-infinite bulk

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Kane-Mele spin-orbit coupling opens a topological gap
h.add_rashba(0.1) # break z-mirror symmetry to expose the edge states
(k,e,ds,db) = h.get_surface_kdos(delta=1e-2) # surface and bulk spectral functions
```

`ds` and `db` are, respectively, the surface and bulk spectral weight at each `(k,e)`; plotting `k,e` colored by `ds` shows the topologically-protected edge states living at the boundary, absent from the bulk spectrum `db`. See `examples/readme_examples/surface_2dTI/main.py` for a runnable version.

Twisted bilayer graphene structural relaxation

Rigidly twisting two graphene layers is only an approximation: below a few degrees of twist, the real lattice relaxes so that the energetically costly AA-stacked regions shrink and triangular AB/BA (Bernal) domains grow around them, separated by solitonic domain walls (Nam & Koshino, arXiv:1706.03908). `GrapheneGeometry` wraps any graphene multilayer `Geometry` (bilayer, twisted bilayer, twisted trilayer, ...) and adds a `.relax()` method that reproduces this effect by minimizing a phenomenological energy over an in-plane relaxation

displacement field: the interlayer Generalized Stacking Fault Energy (GSFE, a closed-form periodic function of the local interlayer registry, fit to graphene's AA/AB/BA stacking energies) plus the intralayer linear-elastic energy, both taken from Carr, Massatt, Torrisi, Cazeaux, Luskin, Kaxiras, arXiv:1805.06972, Table 1. The minimization runs entirely in-plane (no out-of-plane corrugation yet) using jax autodiff gradients passed to a scipy L-BFGS-B solver.

`GrapheneHamiltonian` builds the actual tight-binding Hamiltonian from a (relaxed or rigid) `GrapheneGeometry`, defaulting to the same distance-decaying hoppings as `specialhamiltonian.twisted_bilayer_graphene` – since those hoppings depend on the true 3D interatomic distance, the relaxed positions feed into the electronic structure automatically

```
from pyqula import specialgeometry
from pyqula.graphenetk.geometry import GrapheneGeometry
from pyqula.graphenetk.hamiltonian import GrapheneHamiltonian

g0 = specialgeometry.twisted_bilayer(m0=15) # ~2 degree twist
g = GrapheneGeometry(g0).relax() # AA area shrinks, AB/BA domains grow
h = GrapheneHamiltonian(g)
(k,e) = h.get_bands(num_bands=20)
```

See `examples/2d/graphene_relax/main.py` for a runnable version comparing the rigid and relaxed lattices, and `tests/moire/test_graphene_relax.py` for the physical invariants this is checked against (AA is the GSFE maximum and AB/BA the degenerate minima; relaxed bond lengths stay physical; the local relaxation amplitude grows monotonically as the twist angle shrinks).

Topological insulators

Here we provide a discussion of observables related with topological insulators

Topological invariants

Chern number

The Chern number characterizes two-dimensional topological insulators with broken time reversal symmetry. It is defined as

$$C = \frac{1}{2\pi}\int \Omega(\mathbf{k}) d^2\mathbf{k}$$

where Ω is the Berry curvature. The Chern number can be computed with the following code

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
```

```
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
C = h.get_chern() # Chern number
```

Tensor-cross-interpolation (qtci) integration By default the Brillouin-zone integral above is a plain sum over a uniform `nk` x `nk` mesh, so its cost grows as `nk^2` and the accuracy is set by how finely that mesh resolves the Berry curvature. When the curvature is sharply peaked – near a gap closing, or a nearly-flat band – the mesh has to be very fine before the answer settles.

`integration="qtci"` instead evaluates the integral by *quantics tensor cross interpolation*: the integrand is treated as a function on a binary-refined grid and learned adaptively, sampling only where the function actually varies, with Gauss-Kronrod quadrature on the resulting representation. The number of evaluations then grows roughly logarithmically rather than quadratically in the effective resolution.

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
h.add_haldane(0.05)
C = h.get_chern(integration="qtci",nk=20) # tensor-cross-interpolated BZ integral
```

The same backend can compute the density matrix in a mean-field calculation, replacing the k-mesh sum there:

```
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
hscf,e = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,mf="antiferro",
        nk=8,maxerror=1e-4,return_total_energy=True,integration="qtci")
```

This is backed by `qutecipy`, a pure-Python port of `TensorCrossInterpolation.jl` vendored into pyqula at `src/pyqula/qutecipytk/` (MIT, no extra install needed). Both entry points are checked against their plain counterparts – the qtci Chern number against the analytic value on trivial and topological Haldane models, and the qtci density matrix against the full dense one – in `tests/topology/test_haldane_chern.py` and `tests/scf/test_densitydensity_qtci.py`. Runnable versions are in `examples/2d/chern_qtci/main.py` and `examples/2d/mean_field_qtci/main.py`. Note the density-matrix path currently supports 2D Hamiltonians only.

Z2 invariant

The Chern number characterizes two-dimensional topological insulators with time reversal symmetry. It can be computed with the following code

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
```

```
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Add spin-orbit coupling
from pyqula import topology
z2 = topology.z2_invariant(h) # Z2 invariant
```

Quantum geometric tensor (multiorbital/multiband)

The quantum geometric tensor (QGT) generalizes the Berry curvature: its antisymmetric part is the Berry curvature and its symmetric part is the quantum metric, a measure of the distance between neighboring Bloch states. For a chosen band subspace S (e.g. the occupied bands) it is

$$Q_{ij}^{mn}(\mathbf{k}) = \sum_{l \notin S} \frac{\langle u_m|\partial_{k_i} H|u_l\rangle\langle u_l|\partial_{k_j} H|u_n\rangle}{(E_m - E_l)(E_n - E_l)}, \qquad m, n \in S$$

with the quantum metric $g_{ij}^{mn} = \mathrm{Re}\, Q_{ij}^{mn}$ (symmetric part) and Berry curvature $\Omega_{ij}^{mn} = -2\,\mathrm{Im}\, Q_{ij}^{mn}$ (antisymmetric part) recovered in the band-trace ("Abelian") case; setting `non_abelian=True` instead returns the full band-pair-resolved ("non-Abelian") tensor. Because only states *outside* S enter the energy denominators, this stays well defined even when S contains an exactly or nearly degenerate multiplet of bands – e.g. an exactly spin-degenerate pair, or several orbitals meeting at a high-symmetry point – which an ordinary single-band Kubo formula cannot handle; this is what makes it suitable for genuinely multiorbital/multiband tight-binding models, not just single isolated bands. `dH/dk_i` is evaluated with pyqula's exact analytic multicell k-derivative (no finite-difference error), with k in the same reduced (dimensionless, period-1) coordinates as the rest of pyqula's k-space code (e.g. `topology.berry_curvature`) – not Cartesian k, so the quantum metric's absolute scale is reciprocal-lattice-dependent if you convert to Cartesian coordinates yourself. `occ_idxs` defaults to the bands with $E < 0$, the same convention `h.get_chern()` uses, so it tracks `h.shift_fermi(...)`.

```
from pyqula import geometry
from pyqula import topology
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system (spinful by default)
h.add_haldane(0.2) # Add Haldane coupling
h.shift_fermi(0.3) # put the Fermi level safely mid-gap (gap is [-0.9,0.9])

Q = h.get_quantum_geometric_tensor(k=[0.1,0.2,0.],occ_idxs=[0,1]) # at a k-point
g_metric = h.get_quantum_metric(k=[0.1,0.2,0.],occ_idxs=[0,1]) # quantum metric only

# band-pair-resolved (non-Abelian) tensor of the two occupied bands
Qna = topology.quantum_geometric_tensor(h,k=[0.1,0.2,0.],occ_idxs=[0,1],
        non_abelian=True)

# along a k-path, and integrated over the BZ (cross-checked against
```

```
# the independent Wilson-loop h.get_chern() in tests/topology/test_quantum_geometric_tensor
inds,gpath,omegapath = topology.quantum_geometric_tensor_path(h,occ_idxs=[0,1])
C = topology.chern_from_qgt(h,nk=20,occ_idxs=[0,1])
```

See `examples/2d/quantum_geometric_tensor/main.py` for a runnable version, and `src/pyqula/topologytk/qgt.py` for the implementation and references. There is also an older, unrelated Green's-function/Kubo estimator of the whole-occupied-manifold quantum geometry trace (not band- or band-pair-resolved), `pyqula.topologytk.quantumgeometry.get_QG_kpath`, see `examples/2d/quantum_geometry/main.py`.

Berry curvature density in frequency space

The berry curvature in frequency space is defined as

$$\Omega(\mathbf{k}) = \int_{-\infty}^{\epsilon_F} \Xi(\mathbf{k},\omega) d\omega$$

where Ω is the Berry curvature of the occupiad bands and $\Xi(\mathbf{k},\omega)$ is the energy-resolved Berry curvature. `topology.chern_density` integrates $\Xi(\mathbf{k},\omega)$ over the whole Brillouin zone at a set of energies, giving the frequency-resolved Berry-curvature density and its cumulative (energy-integrated) sum

```
from pyqula import geometry
from pyqula import topology
import numpy as np
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(es,cs,csi) = topology.chern_density(h,nk=10,es=np.linspace(-1.0,1.0,40))
```

`es` are the energies, `cs` the Berry-curvature density at each energy, and `csi` its cumulative integral. In the gap, `csi` should plateau at a value related to the total Chern number of the occupied bands, but this Green's-function-based estimator is numerically delicate (it involves a finite-difference k-derivative, so it needs a fine enough `nk`/`dk` and a large enough `delta` to avoid spurious peaks near quasi-degenerate k-points) and its overall sign/normalization is not guaranteed to match `h.get_chern()` – treat it as a qualitative frequency-resolved profile and cross-check any quantitative reading against `h.get_chern()`. The k-resolved counterpart at a single energy, $\Xi(\mathbf{k},\omega)$ over a full k-mesh, can be obtained with `topology.dOmega_dE_kmap(h,nk=40)`, which writes the map to `BERRY_DENSITY_KMAP.OUT`. See `examples/2d/berry_density_kmap/main.py` for a runnable version.

Berry curvature density in real-space

The berry curvature in real-space is defined as

$$\Omega(\mathbf{k}) = \int \Gamma(\mathbf{k}, \mathbf{r}) d^2\mathbf{r}$$

where Ω is the Berry curvature of the occupiad bands and $\Gamma(\mathbf{k}, \mathbf{r})$ is the spatially-resolved Berry curvature. Note that this object is meaningful for periodic systems with very large unit cells. In pyqula, the real-space Berry curvature is obtained from a Bianco-Resta-type commutator of position and projector operators, evaluated on a large real-space (0-dimensional) supercell or island with `topology.real_space_chern`

```
from pyqula import islands
from pyqula import topology
g = islands.get_geometry(name="honeycomb",n=8,nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(r,c) = topology.real_space_chern(h) # spatially-resolved Berry curvature
```

`r` are the site positions and `c` the local Berry-curvature marker at each site (the call also writes `REAL_SPACE_CHERN.OUT`). See `examples/0d/real_space_chern/main.py` for a runnable version.

Chern number in real-space

The berry curvature in real-space is defined as

$$C = \int F(\mathbf{r}) d^2\mathbf{r}$$

where C is the total Chern number of the occupiad bands and $F(\mathbf{r})$ is the spatially-resolved Chern number. Note that this object is meaningful for periodic systems with very large unit cells. $F(\mathbf{r})$ is the local marker computed by `topology.real_space_chern`; because it is built from a commutator $C = PXPYP - PYPXP$, its trace over the *entire* finite sample is exactly zero by construction, so summing it over every site is not how the invariant is recovered. Instead, deep in the interior of a large enough island – away from the boundary, where the local environment looks like the infinite periodic bulk – the marker plateaus at (approximately) the quantized bulk Chern number, while the edge sites carry compensating opposite-sign weight that cancels the bulk contribution exactly. This exact real-space cancellation is itself a manifestation of the bulk-boundary correspondence

```
from pyqula import islands
from pyqula import topology
import numpy as np
g = islands.get_geometry(name="honeycomb",n=8,nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
```

```
(r,c) = topology.real_space_chern(h) # local marker, per site
r = np.array(r)
bulk = np.argsort(np.linalg.norm(r-r.mean(axis=0),axis=1))[:len(r)//4] # innermost sites
C = np.mean(c[bulk]) # bulk plateau value approximates the total Chern number
```

Topological surface states

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
kdos.surface(h) # surface spectral function
```

Topological markers

The real-space Berry curvature/Chern marker of the two sections above is an example of a topological marker: a local, position-resolved quantity, computable from ground-state projectors alone, that reveals a bulk topological invariant without relying on translational symmetry or a clean Brillouin zone. This makes topological markers well suited to disordered systems, finite flakes and islands, or systems with spatially varying parameters (e.g. a Haldane mass that changes sign across a boundary, or a topological insulator with dilute vacancies), where the marker density directly visualizes where the invariant is carried. See `topology.real_space_chern` above for the code that computes it.

Response functions

Here we discuss how response functions can be computed

Charge-charge response function

The charge-charge response function for a spinless system is computed as

$$\chi(\omega, i, j) = \sum_{n,m} f(\epsilon_n)(1 - f(\epsilon_m)) \frac{\Psi_n(i)\Psi_m(j)\Psi_m^*(i)\Psi_m^*(j)}{\epsilon_n - \epsilon_m - \omega + i\delta}$$

where $f(\epsilon)$ is the Fermi-Dirac distribution

```
from pyqula import geometry
g = geometry.chain() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
(es,chis) = h.get_chi(q=[0.,0.,0.]) # get response function
```

Optional arguments - q: momentum transfer of the response function - energies: array of frequencies - delta: broadening - nk: number of k-points used in the Brillouin-zone integration

The charge-charge response can also be scanned over momentum transfer to build a $\chi(q,\omega)$ map

```
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian()
qs = np.linspace(-1.,1.,40)
chis = [h.get_chi(q=[q,0.,0.],energies=np.linspace(-3,3,100),nk=40,delta=0.1)[1] for q in qs
```

See `examples/1d/charge_response/main.py` for a runnable version.

Generic operator-operator response function

`h.get_chi` computes the charge-charge response by default, but any pair of operators can be used instead, giving the generalized response

$$\chi_{AB}(\omega,q)=\sum_{k,n,m} f(\epsilon_{k,n})(1-f(\epsilon_{k+q,m}))\frac{\langle\Psi_{k,n}|A|\Psi_{k+q,m}\rangle\langle\Psi_{k+q,m}|B|\Psi_{k,n}\rangle}{\epsilon_{k,n}-\epsilon_{k+q,m}-\omega+i\delta}$$

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True)
sz = h.get_operator("sz") # any of the operators from the "Operators" section
(es,chi) = h.get_chi(A=sz,B=sz,energies=np.linspace(-2.,2.,100),nk=40,delta=0.1)
```

By default `A=B=`identity, which recovers the charge-charge response above. Any operator from the "Operators" section (spin, valley, sublattice, location, Nambu...) can be plugged in to build the corresponding susceptibility.

RKKY response function

The RKKY (Ruderman-Kittel-Kasuya-Yosida) interaction between two magnetic impurities is the effective exchange coupling mediated by the itinerant electrons, and can be obtained from the same non-interacting response-function machinery. `rkky.rkky_map` computes it between a reference site and every other site in the system, as a function of distance

```
from pyqula import geometry
from pyqula import rkky
g = geometry.chain()
h = g.get_hamiltonian()
h.add_onsite(1.)
m = rkky.rkky_map(h,n=10,mode="LR",nk=200) # linear-response RKKY vs distance
```

Optional arguments - mode: `"LR"` (linear-response theory, using the same machinery as `get_chi`) or `"pm"` ("poor man's", computed by explicitly adding a

small magnetic perturbation at each site and evaluating the total energy change) - n: how many neighboring cells/distances to compute - nk: number of k-points used in the Brillouin-zone integration

`m` is an array whose columns are `(distance, ..., ..., RKKY energy)`; the RKKY energy is in the last column

```
x,e = m[:,0],m[:,3]
```

See `examples/1d/RKKY/main.py` and `examples/1d/rkky_minimal/main.py` (which compares `"pm"` and `"LR"` on the same system) for runnable versions.

Spin susceptibility and RPA

The methods above compute the bare (non-interacting) response. For an interacting system, `h.get_spinchi_ladder` and `h.get_spinchi_full` dress the same response with the random phase approximation (RPA), using the Hubbard `U` stored on a mean-field Hamiltonian (`h.V`, see "Interactions at the mean-field level"), giving the physically relevant spin-excitation spectrum of e.g. a magnetically ordered state

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian(has_spin=True)
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf=seed,nk=100) # magnetic state
(es,chis) = hmf.get_spinchi_ladder(energies=np.linspace(0.,2.,100),q=[0.1,0.,0.],nk=40,delta
```

- `get_spinchi_ladder` computes the transverse (S^+/S^-) response, i.e. spin-wave-like excitations
- `get_spinchi_full` computes the full (S_x, S_y, S_z) tensor response
- `RPA=True` (default) dresses the response with the interaction; `RPA=False` returns the bare response
- `h.get_qdos_iets` scans `get_spinchi_full` over a q-path instead of a single q, directly giving a spin-excitation dispersion map along high-symmetry directions (needs a 2D lattice for the `"G","K","M"`-style path labels)

```
g2 = geometry.honeycomb_lattice() # already two sublattices per cell
h2 = g2.get_hamiltonian(has_spin=True)
hmf2 = h2.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf="antiferro")
qdisp = hmf2.get_qdos_iets(energies=np.linspace(0.,2.,60),qpath=["G","K","M"],nq=30,nk=20,de
```

- `h.get_iets_ldos` instead computes the spatially resolved response at a single energy, i.e. a real-space map of the spin-flip ("inelastic tunneling spectroscopy", IETS) signal, which combined with `h.get_ldos` gives elastic + inelastic STM-like maps

Both snippets depend on the mean-field state actually being ordered, which is worth checking with `hmf.get_vev("sz")` before reading anything into the

excitation spectrum: an RPA calculation on top of an unpolarized reference state runs perfectly happily and tells you nothing about magnons. Two things decide it. The unit cell must be able to hold the order – a one-site `geometry.chain()` cell cannot represent Neel order at all, and seeding it with `mf="antiferro"` converges to exactly zero moment; `bichain` and `honeycomb_lattice` both have the two sublattices. And `U` must be past the ordering transition: on honeycomb at half filling `U=2` gives a moment of 0.002 (i.e. none) while `U=3` gives 0.44. Note also that `get_qdos_iets`'s cost is the product of its `nq`, `nk` and energy-grid sizes, so it grows quickly – the grid above takes well under a minute, while a 80x40x100 one is closer to an hour.

See `examples/1d/rpa/main.py` (RPA spin response vs q for an antiferromagnetic chain), `examples/2d/rpa_triangular/main.py`/`examples/2d/rpa_honeycomb/main.py` (`get_qdos_iets` dispersion along a q-path) and `examples/0d/rpa_island/main.py`/`examples/0d/rpa_finite` (`get_iets_ldos` real-space IETS maps) for runnable versions.

RPA kernel poles and magnon bands

The RPA-dressed response $\chi_{RPA} = \chi(1 - U\chi)^{-1}$ diverges wherever the kernel $1 - U\chi(q, \omega)$ becomes singular: these poles are the system's collective modes (spin waves/magnons, plasmons) or, if a kernel eigenvalue crosses zero at $\omega = 0$, a sign of a Stoner/RPA instability. `h.get_rpa_kernel_poles` scans a frequency window at a fixed `q` and returns every such pole, generalizing `chi_AB_RPA` (any operators `A`,`B` and interaction matrix `V`, defaulting to the charge channel and $q = 0$ like the other generic response functions above):

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian()
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,nk=100,mf=seed,filling=0.5)
N = len(g.r) # the response matrix is one entry per site
V = 3.0*np.identity(N) # charge-channel interaction, in site space
poles = hmf.get_rpa_kernel_poles(V=V,q=[0.1,0.,0.],
        energies=np.linspace(0.,4.,200),delta=2e-2,nk=40)
```

Two things there are easy to get wrong. The mean field needs a unit cell that can *hold* the order being sought – a one-site `geometry.chain()` cell cannot represent Neel order at all, so seeding antiferromagnetism on it is meaningless; `bichain` gives the two sublattices, and the converged state has `hmf.get_vev("sz")` equal and opposite on them. And `V` must live in the same space as the response matrix: `get_rpa_kernel_poles` defaults to the charge channel, one entry per *site*, so `V` is `NxN`. It is **not** `hmf.V`, which is the mean-field interaction in spin-orbital space (`2Nx2N`) and raises a dimension error here. For the spin channel use `get_magnon_bands` below, which builds the S_x, S_y, S_z vertex from `hmf.V` itself.

`poles` is an `(npoles,2)` array, one row per collective mode found: the pole

frequency and its residual imaginary part. The latter is signed (it is the kernel eigenvalue's actual imaginary part at the crossing, which can lie on either side of the real axis) – judge how sharp/well-defined a mode is by its *magnitude*: small `abs(gamma)` means a sharp mode, large `abs(gamma)` means it is heavily damped or the crossing is numerical noise.

`h.get_magnon_bands` specializes this to the full spin channel used by `get_spinchi_full`/`get_iets_ldos` (the S_x, S_y, S_z tensor, with `U` taken automatically from the mean-field `h.V`, same convention as `get_spinchi_full`) and scans it along a q-path, directly giving the magnon dispersion of a magnetically ordered mean-field state:

```
qs,ws,gammas = hmf.get_magnon_bands(nq=40,energies=np.linspace(0.01,3.,200),delta=2e-2,nk=40
```

Since different q-points can have a different number of poles, `qs`,`ws`,`gammas` are flat 1D arrays of equal length ready for a scatter-style dispersion plot – `qs` holds the integer index of the q-point along the path (the same convention `get_bands` uses for its k-axis), `ws` the pole frequency and `gammas` its (signed) residual imaginary part, so filtering `np.abs(gammas) < threshold` keeps only the sharp, well-defined branches. See `examples/1d/magnon_bands/main.py` for a runnable version (an antiferromagnetic Hubbard chain, showing both its acoustic and optical magnon branches).

Interactions beyond onsite

The `V` passed to `get_rpa_kernel_poles` is not restricted to a single onsite matrix: it can also be a real-space hopping-like dictionary `{(n1,n2,n3): matrix}`, keyed by lattice-vector offset in the same convention as `h.get_hopping_dict()`, for an interaction with support beyond the same unit cell. It is Fourier-transformed to $V(q)$ at whatever `q` the response is evaluated at, using the same Bloch-phase convention as the Hamiltonian's own hoppings – an extended interaction is dressed exactly like an extended hopping. For a nearest-neighbor V_1 on a chain this gives the expected $V(q) = 2V_1 \cos 2\pi q$, so the interaction is repulsive at $q = 0$ and attractive at the zone boundary:

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=False)
N = h.intra.shape[0] # one site per cell here
I = np.identity(N)
V = {(0,0,0): 0.0*I, (1,0,0): 0.6*I, (-1,0,0): 0.6*I} # nearest-neighbor V1
poles = h.get_rpa_kernel_poles(V=V,q=[0.5,0.,0.],energies=np.linspace(0.,2.,60),
                               delta=2e-2,nk=200)
```

Note the channel: `get_rpa_kernel_poles` dresses the **charge** response by default, so a dictionary `V` here must be a density-density interaction in site space, of the same dimension as the response matrix `chiAB(...,mode="matrix")`

returns. It is *not* the place to put a spin vertex.

The **spin** channel deliberately does not accept a non-onsite interaction. `get_magnon_bands`, `get_spinchi_full` and `get_spinchi_ladder` read the mean-field interaction from `h.V`, and raise a `ValueError` if it has any key other than `(0,0,0)` – which is exactly what `h.V` looks like after a `VJinteraction` run with a neighbor-shell `J1`/`V1`. The gate is deliberate: the non-onsite spin vertex is not validated (see `chitk.spinchi._require_onsite_only_V`'s docstring for the caveats), so the library refuses rather than returning a plausible-looking wrong dispersion. To work with a non-onsite spin interaction anyway, build the vertex explicitly and call `chitk.rpa.rpa_kernel_poles_ops`/`chi_ops_RPA` directly, bypassing `h.V` – `tests/chi/test_rpa_nononsite_interaction.py` and `tests/scf/test_rpa_nononsite_ferro_chain.py` do exactly that, and are worked examples of the low-filling ferromagnetic (Stoner) instability a nearest-neighbor exchange drives on a chain.

Density (charge) response

`h.get_densitychi_RPA` and `h.get_plasmon_bands` are the density-density (charge) channel analogs of `get_spinchi_full`/`get_magnon_bands`, for a `V1`/`V2`/`V3`-neighbor-shell (+ onsite `U`, + a general `Vr(r)`) density-density interaction, same convention as `Vinteraction`/`VJinteraction`. Unlike the spin-channel functions, they take the interaction directly as parameters instead of reading it from `h.V`, so no mean-field convergence is needed first – they dress the bare susceptibility of whatever Hamiltonian is passed in (which can also be an already-converged one, if the RPA response about that reference state is wanted):

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True) # 1D chain at half filling
qs,ws,gammas = h.get_plasmon_bands(V1=0.6,qpath=[[0.3,0.,0.],[0.4,0.,0.],[0.5,0.,0.]],nq=3,
                                   energies=np.linspace(0.,1.,100),delta=2e-2,nk=2000)
```

A 1D chain at half filling has perfect Fermi-surface nesting at $q = \pi$, strongly enhancing the static charge susceptibility there – the charge-channel analog of the low-filling ferromagnetic instability above, driven by a repulsive `V1` instead (see `tests/chi/test_plasmon_bands.py`).

Quantum transport

In this section we discuss how we can perform quantum transport calculations with pyqula.

Magnetoresistance in metal-metal transport

As specific example, here we will address how we can compute magnetoresistance in transport between two magnetic metals. We build two copies of the same lead, give each one an exchange field pointing in a different direction, and compare the conductance of the parallel and antiparallel configurations

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
es = np.linspace(-.5,.5,50) # set of energies for dIdV
Gs = dict()
for name,m2 in [("parallel",[0.,0.,0.5]),("antiparallel",[0.,0.,-0.5])]:
    h1 = h.copy() ; h1.add_exchange([0.,0.,0.5]) # first lead, fixed magnetization
    h2 = h.copy() ; h2.add_exchange(m2) # second lead, parallel or antiparallel
    HT = heterostructures.create_leads_and_central(h1,h2,h1) # create the junction
    Gs[name] = [HT.didv(energy=e) for e in es] # calculate conductance
```

The magnetoresistance follows from the two conductance curves, e.g. MR $= (G_P - G_{AP})/G_{AP}$ evaluated at the Fermi energy.

Superconductor-metal transport

Here we address how transport between a superconducting lead and a metallic lead can be computed. As paradigmatic example, we will focus on the Andreev reflection regime and the tunneling regime

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() # first lead
h2 = h.copy() # second lead
h2.add_swave(.01) # the second lead is superconducting
es = np.linspace(-.03,.03,100) # set of energies for dIdV
for T in np.linspace(1e-3,1.0,6): # loop over transparencies
    HT = heterostructures.build(h1,h2) # create the junction
    HT.set_coupling(T) # set the coupling between the leads
    Gs = [HT.didv(energy=e) for e in es] # calculate conductance
```

Transport through an arbitrary finite region

The two examples above build the central scattering region out of copies of the leads' own unit cell. `Hamiltonian.get_central_heterostructure(i,j,left=None,right=None)`

instead lets *any* finite (0d) Hamiltonian act as the central region, contacted by two semi-infinite 1D chain leads attached at sites `i` and `j`:

```
from pyqula import geometry

g = geometry.chain()
gc = g.get_supercell(5)
gc.dimensionality = 0 # a finite, 5-site cluster (no periodicity)
hc = gc.get_hamiltonian() # the central region -- can be any 0d Hamiltonian

h_normal = g.get_hamiltonian() # normal lead
h_sc = g.get_hamiltonian(); h_sc.add_swave(0.05) # superconducting lead

ht = hc.get_central_heterostructure(0,4,left=h_normal,right=h_sc)
G = ht.didv(energy=0.02) # Andreev conductance, via the BdG scattering-matrix formula
```

It returns a plain `Heterostructure`, so every existing method (`landauer`, `didv`, `get_dos`, `get_kappa`...) applies unmodified. `left`/`right` default to a plain spinless chain when omitted, and `j` defaults to the last site. At most one of `{hc, left, right}` may carry an actual pairing amplitude – e.g. a normal lead + normal lead + superconducting central region (a proximitized molecule) is fine, as is the normal + superconducting lead case above, but two superconducting leads raise a `ValueError` (there would be no normal lead left to define a reflection amplitude against; use `heterostructures.build` + `get_dc_current` for that case instead, see below). Only 0d central regions are supported so far. See `examples/transport/central_region_ij/main.py` for a runnable script.

Multiple Andreev reflection and AC-Josephson current

`didv`/`landauer` above are equilibrium, zero-bias linear-response quantities. For a voltage-biased junction between two superconductors (an SNS junction), a finite bias makes each lead's pairing phase wind in time, giving rise to multiple Andreev reflections (MAR) and an AC Josephson effect; the physically meaningful, measurable quantity is the time-averaged (DC) current $I_{dc}(V)$. `Heterostructure.get_dc_current(voltage)` computes this with the Floquet-Keldysh formalism of San-Jose, Cayao, Prada and Aguado, *New J. Phys.* **15**, 075019 (2013) (arXiv:1301.4408): the bias is gauged away from the (static) leads into a single, periodically time-dependent "weak link" hopping, and the resulting Floquet-space Dyson/Keldysh equations are solved to get $I_{dc}(V)$. It works for any combination of normal and superconducting leads – including the case of **two** superconducting leads, which the scattering-matrix formula behind `didv` cannot handle on its own (it has no normal lead to define a reflection amplitude against).

`didv` takes this formalism as an optional `method` argument, so a linear-response conductance can be obtained without calling `get_dc_current`/differentiating by hand: `method="smatrix"` is the original zero-temperature scattering-

matrix/BdG conductance; `method="keldysh"` returns dI/dV at bias `energy`, computed as a central finite difference of `get_dc_current` (`HT.didv(energy=v, method="keldysh", dv=..., nmax=..., temperature=...)`, extra keyword arguments are forwarded to `get_dc_current`); the default, `method="auto"`, picks `"keldysh"` when both leads carry an actual (nonzero) pairing amplitude and `"smatrix"` otherwise, since a single/no superconducting lead is already handled exactly by the scattering matrix.

"Works for any combination of leads" means `get_dc_current` runs without error for a mixed normal/superconducting **Heterostructure** junction too – but its result is **not** directly comparable to `didv(method="smatrix")` on the same junction, even in the limit where the superconducting lead's pairing amplitude is taken to zero (this caveat is specific to `Heterostructure`; see below for why `LocalProbe` is different). The two methods use different bias conventions: `smatrix` freezes the normal lead's self-energy at absolute energy 0 (a grounded, wide-band probe with the entire bias dropped on the other lead), while `dc_current` evaluates *both* leads' self-energies at their actual Floquet sideband energies (needed for, and validated against, a normal-normal rigid two-terminal bias reference in `tests/keldysh`). Forcing `method="keldysh"` where `method="auto"` would have picked `"smatrix"` therefore computes a different physical quantity, not a slower/more-general version of the same one – confirmed by direct comparison to disagree by an O(1), non-vanishing factor as the extra lead's pairing amplitude shrinks to zero. Exactly at sub-gap bias this is compounded further by real (not numerical) physics: an infinitesimal pairing amplitude still opens a hard gap exactly at the Fermi level, which `get_dc_current`'s sideband ladder always samples (its quasienergy integral starts at 0), so its zero-pairing limit at that energy does not equal the exactly-normal-lead value there either.

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() ; h1.add_swave(0.1) # left superconducting lead
h2 = h.copy() ; h2.add_swave(0.1) # right superconducting lead
HT = heterostructures.build(h1,h2) # create the SNS junction
HT.set_coupling(0.5) # set the normal transparency of the weak link
vs = np.linspace(0.02,1.5,40)*0.1 # bias voltages
Is = HT.get_iv_curve(vs) # MAR/AC-Josephson dc current
```

Budget for the cost: this is the most expensive family in the guide, at roughly 15 seconds per bias point for a two-lead SNS junction, so the 40-point curve above is several minutes. Note also the bottom of that voltage grid. The number of sidebands needed grows as the bias falls (the MAR order $2\Delta/eV$ does), so a grid reaching very close to zero bias – `0.02*delta` here – can genuinely exhaust `nmax_max` and warn that "sidebands did not converge", meaning that point is

inaccurate rather than that anything crashed. Raising the grid's lower endpoint (to `0.15*delta`, say) is the cheap fix; raising `nmax_max` is the expensive one.

The number of Floquet sidebands is increased adaptively (as in the paper) until I_{dc} converges; see `examples/transport/floquet_keldysh_mar/main.py` for a runnable script and `tests/keldysh/` for correctness tests (a normal-normal junction must reduce exactly to a directly biased, non-Floquet Landauer calculation, and a normal-superconductor junction's zero-bias slope must match the existing equilibrium Andreev conductance from `didv`). Only 1D leads are supported. An explicit central region (`heterostructures.build(h1,h2,central=[hc])`, e.g. a quantum dot detuned from the leads) works too, solved through the general dense Floquet inversion rather than the fast two-block chain decomposition – correspondingly slower, since the whole (block x sideband) matrix is inverted at every quasienergy. Note where the bias is assumed to drop: the AC-carrying bond is the junction's rightmost one, so the central region sits at the **left** lead's electrostatic potential. That is a physical model choice, not a gauge choice – a comparison against a static-bias reference has to shift the central region along with the left lead, and a central Hamiltonian must be a valid BdG (particle-hole symmetric) one, so detune it with `hc.shift_fermi(eps)` rather than by adding `eps` to the diagonal of `hc.intra`.

`transporttk.localprobe.LocalProbe` models a single STM-like tip weakly coupled to one site of an infinite/bulk sample (used e.g. for `get_kappa`, a decay-constant/transparency-scaling diagnostic – see `examples/transport/decay_constant/main.py`). The same routing applies there: `LocalProbe.didv/get_kappa` use the ordinary scattering-matrix formula by default, but switch to the Floquet-Keldysh MAR current when the probe lead (`lp.lead`) is itself superconducting *and* the sample is superconducting too, since a normal-metal probe no longer applies and the same "no normal lead to reflect against" problem as above appears. The probe's unit cell and the sample's local (single-site) Hamiltonian play the role of the two leads.

Unlike `Heterostructure`, forcing `method="keldysh"` on a `LocalProbe` whose probe is normal (or negligibly paired) *is* consistent with `method="smatrix"`: `LocalProbe`'s Keldysh path grounds a normal probe lead (freezes its self-energy at absolute energy 0, `smatrix`'s own convention for it) precisely so the two agree – exactly in the wide-band-lead limit, to within a few percent for a probe with genuine band structure over the bias window (e.g. a plain tight-binding chain). This grounding only applies when the probe lead itself has no real pairing; when it does (the case below), the probe's self-energy is evaluated at its actual Floquet sideband energy instead, exactly as for `Heterostructure` – grounding a genuinely superconducting probe would pin every evaluation at its own gap center and was confirmed to suppress the MAR current by over an order of magnitude, so the two-superconductor case keeps working as originally validated.

```
from pyqula import geometry
from pyqula.transporttk.localprobe import LocalProbe
```

```
g = geometry.chain()
h = g.get_hamiltonian() ; h.shift_fermi(1.) ; h.add_swave(0.1) # SC sample
lead = geometry.chain().get_hamiltonian() ; lead.shift_fermi(1.) ; lead.add_swave(0.1) # SC
lp = LocalProbe(h,lead=lead,delta=1e-3)
lp.T = 0.3 # reference transparency
G = lp.didv(energy=0.25,nmax=4,nmax_max=12,tol=5e-2) # routed through Keldysh automatically
k = lp.get_kappa(energy=0.25,nmax=4,nmax_max=12,tol=5e-2)
```

This is considerably more expensive than the normal-probe case (each `didv`/`get_kappa` call runs several Floquet-Keldysh sideband sweeps), especially deep below the combined gap at low transparency, where the sideband sum converges slowly; see `examples/transport/decay_constant_keldysh/main.py` for a runnable script using a coarse energy grid and a modest sideband cutoff to keep the runtime reasonable.

`get_kappa` also accepts a `temp` argument for a thermally-averaged kappa (each conductance entering the power-law fit is `didv(temp=...)`'s thermal average rather than the zero-temperature value); `temp=0` (the default) is unchanged. Pass a single `energy` (returns a scalar, as above) or a whole `energies=[...]` array at once (returns an array):

```
k = lp.get_kappa(energies=[0.1,0.25,0.4],temp=0.02,nmax=4,nmax_max=12,tol=5e-2)
```

`Heterostructure.get_kappa` takes the same `temp`/`energies` arguments.

A single `get_dc_current`/`keldysh_didv`/`get_kappa` call solves each lead's Sancho-Rubio/`bloch_selfenergy` self-energy directly at every energy the sideband sweep visits by default (`selfenergy_method="direct"`), since building an AAA interpolant (many true solves at increasingly refined candidate energies) usually costs more than that one `"direct"` call by itself – the win comes from reuse, not the first call. The opt-in `selfenergy_method="aaa"` (`use_aaa=True` for `didv`/`keldysh_didv`) instead replaces most of those many-thousands of individual solves with evaluations of a compact rational (AAA) interpolant built from far fewer true solves (`keldyshtk.current.build_selfenergy_aaa`).

A **sweep** over many calls is exactly the workload that reuse pays for, so the three sweep entry points default the OTHER way, to `"aaa"`/`use_aaa=True`, building and sharing one interpolant across the whole sweep instead of leaving each call to independently build (and discard) its own: `get_iv_curve` (a `get_dc_current` voltage sweep), `didv(energies=...)`/`didv_curve` (a `didv` energy sweep), and `get_kappa(energies=...)`'s finite-temperature path (whose internal thermal quadrature alone can visit well over a hundred nearby energies for just one nominal `(energy, temp)` point). Pass `selfenergy_method="direct"` (or `use_aaa=False`) explicitly to any of these to opt back out. Each falls back to `"direct"` automatically, per-sweep, if the shared fit doesn't converge within its budget.

An earlier version of this had a real accuracy gap, growing with the sideband window (`nmax_max`) – up to ~10% relative current error in the worst case

investigated. `documentation/keldysh_aaa_selfenergy_accuracy_plan.md` root-caused this to the interpolant's candidate grid being under-resolved (both at a lead's own gap-edge singularities and, more consequentially for the current-error trend, across the fit's broader domain) in a way the interpolant's own held-out validation check – confined too close to existing candidates – never detected. Both the validation sampling and the grid-refinement strategy (`aaatk.selfenergy_aaa._refine_grid`) were fixed and validated directly against the current (not just the self-energy fit) across the same `nmax_max` sweep that exposed the original gap: relative current error is now consistently under ~1% throughout, with no growth trend (see that document's closing update for the full measurement). `selfenergy_method="aaa"` still checks its own convergence (`.converged`) and safely falls back to `"direct"` if a fit can't reach its target tolerance within a bounded budget, rather than silently returning an under-resolved answer – but as with any interpolation-based shortcut, checking agreement with `"direct"` for your own system/parameter range before relying on it is still good practice.

`didv`'s `energy` and `energies` arguments are mutually exclusive, the same convention `get_kappa` already uses: pass a single `energy` (returns a scalar) or a whole `energies=[...]` array at once (returns an array, and internally dispatches to `didv_curve(ht, energies, **kwargs)`). Passing an array directly as the scalar `energy=` is not supported (it fails, a numpy broadcasting error on the `smatrix` path, an "ambiguous truth value" error on the `keldysh` path) – every example that plots dI/dV vs. energy predates `energies=` and loops explicitly instead, `[ht.didv(energy=e) for e in es]`, which still works but is no longer necessary. `energies=` additionally – like `get_iv_curve` does for `get_dc_current` – builds and shares ONE AAA interpolant across the whole sweep by default (see above), instead of a raw loop's `didv(energy=e, use_aaa=True)` independently building (and discarding) its own interpolant at every single energy:

```
es = np.linspace(0.15,0.25,40)*0.1 # bias energies
Gs = HT.didv(energies=es, nmax_max=40) # shared-AAA dI/dV curve (use_aaa=True is now the de
```

`HT.didv_curve(es, **kwargs)`/`lp.didv_curve(es, **kwargs)` are the same thing called directly, for a caller who wants the array entry point without going through `didv`.

For a Floquet-Keldysh-eligible junction (both leads, or a `LocalProbe`'s probe+sample, superconducting), `didv(temp=...)`/`get_kappa(temp=...)` now default to evaluating `dc_current`'s own `temperature` parameter directly (2 `dc_current` calls, a central finite difference in bias) rather than the internal thermal quadrature described in the previous paragraph (`keldysh_thermal_mode="convolution"` on `transporttk.thermaldidv.finite_T_didv` recovers the old behavior, still used for non-Keldysh junctions where it is exact). This is not merely a faster way to get the same number: the two compute genuinely different quantities away from `temp->0` (direct broadens each Floquet sideband's own occupation by `temp`; convolution smears the bias

voltage, an n-dependent displacement of the whole sideband ladder) – see `documentation/keldysh_sideband_decimation_plan.md`'s "direct finite-T Keldysh evaluation" entry for the validation and the measured ~100x-plus speedup.

The outer quasienergy integral itself is evaluated with a batched adaptive quadrature (`keldyshtk.quadrature.adaptive_quad_batch`): the same 21-point Gauss-Kronrod rule and embedded QUADPACK error estimator `scipy.integrate.quad` uses, at the same tolerance, but with the refinement loop restructured so that every panel awaiting evaluation in a round is evaluated in a single batched, `numba`-parallel chain solve rather than one scalar Python callback per node. It visits essentially the same nodes as `scipy.integrate.quad` did (measured over a whole `get_dc_current` call: 630 vs 588, 1197 vs 1197, 3906 vs 3906 on three superconducting cases, and 84 vs 210 on a normal junction) while collapsing those hundreds-to-thousands of scalar dispatches into 4-54 batched ones. Together with a companion fix to the self-energy cache's per-energy Python bookkeeping (which profiling exposed as the next bottleneck once the quadrature stopped dominating), that is worth 5.0x-11.8x in wall clock across those four cases, with the returned current unchanged to 1e-16 relative on three of them and 1.5e-6 on the fourth. This is the default; it needs no opt-in, and the previous `scipy.integrate.quad` implementation stays reachable as `quadrature="adaptive_scipy"` (the reference the batched rule is validated against, not a mode to choose on its own).

`dc_current` also takes an opt-in `quadrature` argument for the outer quasienergy integral: `"adaptive"` is the batched adaptive rule just described (the default); `"fixed"` instead evaluates a deterministic, fixed-node composite Gauss-Legendre rule whose node/weight set is a pure function of `voltage` alone (`quad_panel_width`/`quad_min_panels`/`quad_order` control it), known in full before any integrand evaluation and solved with a batched, `numba`-parallel chain solver rather than one Python callback per node. Accuracy is not the concern (validated to within ~6e-4 of a tight reference across a broad SC-SC/normal sweep); speed is case-dependent and often *worse* than adaptive quadrature, since a fixed grid has to be dense enough to resolve a gap-edge singularity wherever it happens to land, with no way to discover at runtime that a given case (e.g. a normal junction with no singularity at all) didn't need that density. `"fixed"` is kept as tested, opt-in infrastructure for callers that specifically need a deterministic/cacheable node set (see `documentation/keldysh_sideband_decimation_plan.md`'s "item 2b"/"item 2c" entries for the full numbers) – not a general replacement for the default `"adaptive"` path.

Experimental: a JAX-differentiable Floquet-Keldysh current

`keldyshtk.current_jax.JaxKeldyshCurrent` (needs the optional `jax` extra: `pip install pyqula[jax]`) is an independent reformulation of `dc_current` for zero-temperature, fixed-sideband-count (`nmax`, not adaptively grown) work:

instead of a central finite difference of two separate `dc_current` calls, it batches the whole quasienergy quadrature into one compiled, `vmap`ped computation and differentiates it directly with `jax.grad`. Reused across many voltages (build once per `(junction, nmax, vmax)` combination, call `.current(v)`/`.didv(v)` many times), this is a genuine reformulation, not a drop-in speedup: measured on the same superconducting-probe `LocalProbe` workload the rest of this section targets, both `.current()` and `.didv()` came out roughly break-even to about 2x slower than the direct path once implemented and benchmarked rigorously (see the module's own docstring for the full story, including two real self-energy numerical-edge-case bugs and one silently-dropped derivative term found and fixed along the way). Kept as tested, documented, opt-in infrastructure – a reformulation that did not pay off for the specific workload it was built for, potentially useful for a different one (a system that converges at a smaller `nmax`, or a workload needing only `current()` and not `didv()`) – not something `didv`/`get_dc_current` route to automatically.

Single defects in infinite systems

A single point defect or impurity embedded in an otherwise infinite, periodic system cannot be handled by a plain supercell calculation without artificially periodizing the defect. `embedding.Embedding` solves this properly with a Green's function embedding technique: it takes the pristine, periodic Hamiltonian `h` and a modified intracell matrix `m` describing the defect (or another `Hamiltonian` from which the modified matrix is taken), and gives access to the observables of the infinite system as perturbed by that single, non-periodic defect

```
from pyqula import geometry
from pyqula import embedding
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # pristine, infinite Hamiltonian
hv = h.copy()
hv.add_onsite(lambda r: 1.0 if r[0]<0.01 else 0.0) # a single-site onsite defect
eb = embedding.Embedding(h,m=hv) # embed the defect in the infinite system
(x,y,d) = eb.get_ldos(energy=0.0,delta=1e-2,nsuper=200,nk=400) # LDOS around the defect
```

`get_ldos` returns the real-space positions and the LDOS profile in a window of `nsuper` unit cells around the defect, showing e.g. Friedel oscillations or bound/in-gap states induced by the impurity (it also writes `LDOS.OUT`; pass `write=False` to suppress that). `eb.get_dos()` gives the total DOS, `eb.multildos()` scans the LDOS over many energies (written to a `MULTILDOS/` folder), and `eb.get_didv()` computes transport through the embedded defect. See `examples/embedding/single_impurity_1D/main.py` and `examples/embedding/honeycomb_vacancy/main.py` for runnable versions, and the other scripts under `examples/embedding/` for further defect scenarios (vacancies, boundaries, Yu-Shiba-Rusinov states, self-consistent defects...).

Wannierization

`h.get_wannier_hamiltonian()` Wannierizes a fixed, contiguous range of a periodic Hamiltonian's bands and returns a new, smaller multicell Hamiltonian whose real-space hoppings exactly reproduce that band subspace on the wannierization k-mesh (there is no band disentanglement yet – the selected range is Wannierized jointly as one group).

As an example, consider a staggered honeycomb lattice, where a sublattice potential opens a gap and gives a genuinely dispersive valence band to Wannierize

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
h.add_onsite([0.8,-0.8]) # sublattice potential opens a gap

# Wannierize just the lowest (valence) band: bands=[0,0] selects band
# index 0 at every k-point of a 12x12 Monkhorst-Pack wannierization mesh
hwan = h.get_wannier_hamiltonian(bands=[0,0],nk=12)

print("Number of Wannier functions:",hwan.intra.shape[0])
print("Wannier centres (Cartesian):\n",hwan.wannier_centres)
print("Wannier spreads:",hwan.wannier_spreads)
print("Total spread Omega:",hwan.wannier_spread_total)
```

The Wannierized Hamiltonian `hwan` behaves like any other pyqula Hamiltonian, so its bands can be compared directly against the original model's

```
(k,e) = h.get_bands(write=False)
(kw,ew) = hwan.get_bands(write=False)
```

Symmetry-enforced Wannierization

Passing `symmetries="auto"` makes `get_wannier_hamiltonian` check, before Wannierizing, that the selected band range is a genuine union of point-group-related multiplets everywhere on the mesh (point-group operations are auto-detected from the geometry+Hamiltonian via `symmetrytk.pointgroup.find_point_group`). A band selection that instead slices through a symmetry-related degeneracy is rejected with a `ValueError` rather than silently returning a mis-symmetrized model. A list of explicit `symmetrytk.pointgroup.SymmetryOperation` can be passed instead of `"auto"` to enforce a specific subgroup.

A good illustration is kagome's flat band: it is exactly degenerate with the dispersive middle band at the K point, so no selection containing only the flat band is a union of whole multiplets – this is the well-known topological obstruction behind kagome's flat band having no symmetric exponentially-localized Wannier function, and the check catches it instead of returning a broken model

```
from pyqula import geometry
```

```
from pyqula.symmetrytk import pointgroup

g = geometry.kagome_lattice()
h = g.get_hamiltonian(has_spin=False)

try:
    h.get_wannier_hamiltonian(bands=[2,2],nk=12,symmetries="auto")
except ValueError as e:
    print("Flat band alone correctly rejected:",str(e).splitlines()[0])

# the full 3-band manifold has no such obstruction
hwan_sym = h.get_wannier_hamiltonian(bands=[0,2],nk=12,symmetries="auto")
print("Symmetries enforced:",[c.op.name for c in hwan_sym.wannier_symmetries])
```

See `examples/wannier/get_wannier_hamiltonian/main.py` and `examples/wannier/symmetric_wannieriza` for runnable versions of these two examples.

Chebyshev kernel polynomial (KPM) methods

For very large systems, exact diagonalization becomes impractical. Passing `mode="KPM"` to observable methods switches to a stochastic Chebyshev kernel polynomial expansion, which never builds the full spectrum and scales to systems with millions of sites on a single core. It requires a sparse Hamiltonian (`is_sparse=True`)

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
g = g.get_supercell(3000) # a big supercell
g.dimensionality = 0
h = g.get_hamiltonian(is_sparse=True,has_spin=False)
(x,y) = h.get_dos(mode="KPM",
            energies=np.linspace(-3.0,3.0,200), # energies
            delta=1e-4, # effective smearing (~1/npol)
            ntries=10 # number of random vectors for the stochastic trace
            )
```

The same expansion also gives non-local correlators and Green's functions without inverting a matrix, through the lower-level `kpm` module

```
from pyqula import kpm
(x,y) = kpm.dm_ij_energy(h.intra,npol=200,i=0,j=9,ne=1000)
```

See `examples/0d/kpm_dos/main.py` and `examples/0d/kpm_correlator/main.py` for runnable versions, including a comparison of the KPM correlator against the exact Green's function calculation.

The batched Chebyshev-moment kernels underneath KPM (one starting

vector per random try, or per site for a full trace) run on numba by default, but can be dispatched to a GPU through jax instead by passing `kpm_cpugpu="GPU"` down to any KPM entry point – `h.get_dos(mode="KPM", ...)`, `kpm.tdos`/`kpm.pdos`/`kpm.ldos`, or the lower-level `kpm` module functions themselves all forward it:

```
(x,y) = h.get_dos(mode="KPM",
            energies=np.linspace(-3.0,3.0,200),
            delta=1e-4,ntries=10,
            kpm_cpugpu="GPU") # dispatch the Chebyshev moments to a GPU
```

`kpm_cpugpu="GPU"` transparently falls back to running on the CPU (through jax's own CPU backend) if no GPU is visible, so it is always safe to pass even on a machine without one. The batch of starting vectors is sent to the device in fixed-size chunks (`gpu_batch_size`, default 256, currently a reasoned default rather than a GPU-benchmarked one) rather than all at once, so device memory use stays bounded even for a full-space trace over a large system; pass a smaller `gpu_batch_size` to trade fewer, larger device dispatches for a smaller memory footprint. `kpm_prec="single"` (also forwarded the same way) switches the moments to single precision, useful for squeezing more parallelism out of a GPU when double-precision accuracy is not needed.

There are two distinct KPM code paths for an operator-weighted quantity (a projected DOS/spectral function): passing `operator=` reaches the operator-weighted moments directly (`kpm.tdos`/`kpm.pdos`'s `operator=` argument, or `h.get_kdos_bands(mode="KPM", operator=...)`), while `h.get_dos(mode="KPM", operator=...)` instead confines the random starting vectors to the operator's subspace and falls back to the plain (non-operator-weighted) moments internally. `kpm_cpugpu="GPU"` reaches both, but only the first actually exercises the operator-weighted GPU kernel (`kpm_momentsA_batch_gpu`).

Classical spin models

`classicalspin.SpinModel` models classical (non-quantized) spins on a lattice, each parametrized by a pair of angles (θ_i, ϕ_i), interacting through a real-space tensor exchange $\vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ and an optional Zeeman field. Like `LatticeGas`, it reuses `Geometry` for the lattice/neighbor shells but is otherwise independent of the quantum `Hamiltonian` machinery: the energy is evaluated directly from the angles, and the ground state is found by local (gradient-based, via `jax` autodiff) multistart minimization rather than diagonalization – only the Γ point is supported, so incommensurate (e.g. spiral) textures need an explicit supercell

```
from pyqula import geometry
from pyqula import classicalspin

g = geometry.triangular_lattice() # geometrically frustrated lattice
```

```
g = g.get_supercell(3)

sm = classicalspin.SpinModel(g) # classical spin model on this geometry
sm.add_heisenberg(Jij=[1.0]) # first-neighbor antiferromagnetic exchange
sm.minimize_energy(tries=10) # multistart local minimization

mx,my,mz = sm.get_magnetization() # per-site magnetization components
```

`add_heisenberg` builds shell-based isotropic (or, via `Jm=[Jx,Jy,Jz]`, diagonally anisotropic/XXZ) couplings the same way `Geometry.get_hamiltonian(tij=...)` does. `classicalspin.generating_functions(name=...)` returns ready-made two-point coupling functions for other common forms – `"Linear"` (dipolar $1/r^3$), `"RKKYTI"` (RKKY on a topological-insulator surface, PRB 81 233405), `"ZZ"`/`"XYZ"` (Ising/anisotropic-diagonal), `"DM"` (Dzyaloshinskii-Moriya) – to pass into `add_tensor(fun)` (couplings within the home cell) or `add_tensor_2d(fun,ncells=...,vspiral=...)` (also sums periodic images, and can twist the coupling tensor by a per-image angle to embed a spin-spiral wavevector). `get_local_energy()` gives the per-site energy for spatially resolved maps (e.g. of a skyrmion or domain wall), and `classicalspintk.align.most_perp_basis()` rotates a magnetization texture into the frame where it is mostly in-plane, for quiver-style plotting. See `examples/classicalspin/` for runnable demos, including a frustrated triangular-lattice ground state and a modulated-exchange ladder.

Lattice gas models

`latticegas.LatticeGas` models classical, occupation-based (0/1) degrees of freedom on a lattice – e.g. adsorbates, vacancies, or any classical binary order parameter – interacting through a real-space coupling $J_{ij}n_in_j$ and a site-dependent chemical potential μ_in_i. It reuses `Geometry` to define the lattice and its neighbor shells, but is otherwise independent of the quantum `Hamiltonian` machinery: the energy is evaluated directly from the occupation array, and the ground state is searched with a Metropolis-annealed discrete swap optimizer, not diagonalization

```
from pyqula import geometry
from pyqula import supercell
from pyqula import latticegas

g = geometry.triangular_lattice()
g = supercell.turn_orthorhombic(g)
g = g.get_supercell(10)
g.dimensionality = 0

lg = latticegas.LatticeGas(g,filling=1./3.) # 1/3 of the sites randomly occupied
lg.add_interaction(Jij=[1.,1.,1.]) # first, second and third neighbor repulsion
es = lg.optimize_energy(temp=0.5,ntries=1e4) # simulated annealing
```

`lg.den` holds the current 0/1 occupation array and `es` the energy trajectory of the anneal. `get_local_energy()`/`get_local_mu()` give the per-site energy/chemical-potential contribution for the current snapshot, and `get_correlator()`/`get_structure_factor()` give the real-/reciprocal-space density-density correlator – useful for detecting ordered (e.g. striped, honeycomb-vacancy) ground states and their ordering wavevector. `anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule, and `optimize_energy_multistart()` keeps the best of several independent restarts. `optimize_grand_canonical()` switches from fixed-filling swap moves to single-site flips under `lg.mu`, letting the filling itself fluctuate – useful for scanning a phase diagram vs. chemical potential, or for estimating thermodynamic quantities like the specific heat (`get_specific_heat()`/`get_susceptibility()`) from an equilibrium trajectory at fixed temperature. `add_tensor()` adds couplings beyond fixed neighbor shells, and `write()`/`read()` checkpoint a snapshot to/from disk. See `examples/latticegas/` for runnable demos of annealing, local-energy maps, correlators, and grand-canonical sampling.

Ising models

`latticeising.LatticeIsing` models classical Ising spins $s_i \in \{-1, +1\}$ on a lattice, interacting through a real-space coupling $-J_{ij}s_is_j$ and a site-dependent field $-b_is_i$ – the standard textbook Ising Hamiltonian, with $J > 0$ ferromagnetic (favoring alignment). It mirrors `LatticeGas` closely (same `Geometry`-driven pair list, CSR adjacency cache, and Metropolis machinery, several of whose module-level functions – the adjacency builder, `add_tensor()`, `regroup()`, `get_specific_heat()`/`get_susceptibility()` – are reused directly rather than reimplemented) but uses the **opposite** energy sign convention from `LatticeGas` (whose $\sum J_{ij}n_in_j$ has no minus sign, so positive J there is a *repulsion*): with Ising spins, positive J_{ij} in `add_interaction()` means ferromagnetic alignment, matching the literature convention.

```
from pyqula import geometry
from pyqula import latticeising

g = geometry.square_lattice() # bipartite, so ferromagnetic order is not frustrated
g = g.get_supercell(12)
g.dimensionality = 0

li = latticeising.LatticeIsing(g,m=0.0) # random +-1 spins, zero net magnetization
li.add_interaction(Jij=[1.]) # first-neighbor ferromagnetic coupling
es,ms = li.anneal(temps=[3.,1.,0.3,0.1,0.03],ntries=1e4) # simulated annealing
```

`li.s` holds the current ± 1 spin array. `li.optimize_energy()` runs single-spin-flip Metropolis dynamics – the standard Ising Monte Carlo move set, in which the total magnetization is *not* conserved (it fluctuates under `li.b`), so, mirroring `LatticeGas.optimize_grand_canonical()`, it returns `(es, ms)`:

the energy and total magnetization ($\sum_i s_i$) trajectories, the latter usable directly with `latticegas.get_susceptibility()`. `li.optimize_conserved()` instead uses Kawasaki spin-exchange (swap) dynamics, which *does* conserve the total magnetization – the analog of `LatticeGas.optimize_energy()`'s fixed-filling swaps. `li.anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule, and `optimize_energy_multistart()` keeps the best of several independent restarts. `get_local_energy()`/`get_local_field()` give the per-site energy/effective-field for the current snapshot, and `get_correlator()`/`get_structure_factor()` reuse `LatticeGas`'s real-/reciprocal-space correlator machinery directly (it operates on any per-site array, not just 0/1 occupations) to locate ordered (ferromagnetic, checkerboard antiferromagnetic, ...) ground states and their ordering wavevector. Because `li.pairs` lists both directions of every bond (same convention as `LatticeGas`), `get_energy()` is twice the usual sum-over-unordered-bonds convention – e.g. the 2d square-lattice ferromagnet's critical temperature sits near 2×2.269 in these units, not 2.269. See `examples/latticeising/` for runnable demos of annealing, a temperature scan (magnetization and specific heat), and local-energy/local-field maps.

Main functions and methods

Geometry functions and methods

g.get_hamiltonian()

Generate the Hamiltonian from a geometry.

Optional arguments

- tij = [1.0,.0,0.]: List with 1st, 2nd, 3rd nearest neighbor hopping

Returns the Hamiltonian

g.get_supercell()

Generate a supercell

Arguments

- N: size of the supercell to create, number or tuple, or a 3x3 integer matrix M for a general non-diagonal/non-orthogonal supercell (see "Electronic structure folding and unfolding" for how this interacts with `operator="unfold"`)

Optional arguments

- store_primal=False: keep a reference to the primitive-cell geometry on the supercell, needed by `operator="unfold"` (see "Electronic structure folding and unfolding")

Returns a new geometry

Hamiltonian functions and methods

h.get_bands()

Compute band structure

Optional arguments:

- nk = 20: number of k-points
- operator: a single operator, or a list of operators, to compute expectation values for at each eigenstate

Returns kpoint index and energies, plus one extra row per operator if `operator` is given

h.get_kdos_bands()

Compute a k-resolved spectral function (band structure dressed with a projection operator, or an unfolded spectral function) along a k-path.

Optional arguments:

- kpath: k-point path (auto-generated if not given)
- operator=None: operator used to weight the spectral function, e.g. `"unfold"` (see "Electronic structure folding and unfolding")
- energies, delta, nk: frequency range, broadening, k-point density

Returns k-path fraction, energy and spectral weight

h.get_dos()

Compute the density of states.

Optional arguments:

- energies: array with frequencies of the DOS
- delta=0.01: broadening of the DOS

Return energies and DOS

h.get_gap()

Return the indirect gap, i.e. the smallest energy difference between an empty and an occupied state anywhere in the Brillouin zone (the two need not sit at the same k-point). Obtained by numerically minimizing over k rather than by scanning a fixed mesh, so a gap that closes at an incommensurate k-point is not missed.

Optional arguments:

- ntries=1: repeat the minimization this many times from different random starting points and keep the smallest result – worth raising for a band structure with several nearly degenerate minima

Returns a single number, the gap. Zero (up to numerical noise) for a metal or a Dirac semimetal

h.get_bandwidth()

Return the bottom and top of the spectrum, `(emin,emax)` – note this is the pair of band edges, not their difference. Uses the same k-space optimization as `h.get_gap()`, so the edges are the true extrema over the Brillouin zone rather than the extrema of a k-mesh sample

h.get_filling()

Return the fraction of states below zero energy, i.e. the filling measured with the Fermi energy at $E = 0$. Half filling gives 0.5. Use `h.set_filling(nu)` to shift the onsite energy so that a target filling is realized, and this method to check the result

Optional arguments:

- nk: k-point density used to sample the spectrum

h.get_total_energy()

Return the total energy, i.e. the sum of the occupied single-particle eigenvalues. For a mean-field Hamiltonian this is the band energy only – `h.get_mean_field_hamiltonian(...,return_total_energy=True)` returns the interacting total energy including the double-counting correction instead

Optional arguments:

- nk=10: k-point density of the Brillouin-zone sum
- fermi=0.0: energy below which states are counted as occupied
- mode="mesh": k-space sampling; `use_kpm=True` switches to a Chebyshev estimate for large systems

h.get_density_matrix()

Return the full density matrix of the occupied states, as a dense matrix in the same basis as `h.intra`. See "Interactions at the mean-field level" for the k-resolved, hopping-resolved version the self-consistent loops use

h.get_ipr()

Return the inverse participation ratio of every eigenstate, as `(energies,ipr)`. A delocalized state in a system of N sites gives IPR $\sim 1/N$ and a state local-

ized on one site gives IPR ~ 1, so this is the usual diagnostic for Anderson localization or for in-gap bound states. **Finite (0d) systems only** – it raises `NotImplementedError` for a periodic Hamiltonian; for those use the IPR operator instead (see "Inverse participation ratio operator")

h.get_vev() / h.get_single_vev() / h.get_several_vev()

Ground-state expectation values of operators, evaluated by summing over the occupied states.

- `h.get_vev(operator=...)` returns one real number **per site**: the site-resolved expectation value of `operator` (any name accepted by `h.get_operator`, e.g. `"sz"`), or the site occupation if `operator` is omitted. This is what produces a magnetization or charge-density map
- `h.get_single_vev(A)` returns the single number $\langle A \rangle$ for one operator `A`, summed over the whole system
- `h.get_several_vev([A,B,...])` does the same for a list of operators in one pass, sharing the diagonalization

Optional arguments:

- nk=30: k-point density of the Brillouin-zone sum

h.add_soc()

Add Kane-Mele intrinsic spin-orbit coupling

Arguments:

- value: value of the SOC

h.add_zeeman()

Add a Zeeman field to the Hamiltonian

Arguments:

- value: value of the Zeeman, as a number (assumes [0,0,Bz]), array or callable function

h.add_rashba()

Add Rashba spin-orbit coupling

Arguments:

- value: value of the Rashba SOC

h.add_onsite()

Add a local onsite energy

Arguments:

- value: value of the onsite energy

h.get_ldos()

Compute the local density of states.

Optional arguments:

- e: energy of the LDOS
- delta=0.01: broadening of the LDOS
- projection="TB": `"TB"`, `"TBRS"` (real-space interpolated) or `"atomic"`

Return x, position, y position and LDOS

h.get_multildos()

Compute the LDOS at many energies, writing one file per energy to a `MULTILDOS/` folder.

Optional arguments:

- energies=linspace(-1,1,100): energies to compute
- projection="TB": `"TB"` or `"atomic"`

h.get_chi()

Compute a non-interacting operator-operator response function (charge-charge by default).

Optional arguments:

- q=[0,0,0]: momentum transfer
- A=None, B=None: operators defining the response (default: identity, i.e. charge-charge)
- energies, delta, nk: frequency range, broadening, k-mesh density

Returns energies and the response function

h.get_spinchi_ladder()

Compute the transverse (S^+/S^-) spin susceptibility, RPA-dressed by default using the Hubbard `U` of a mean-field Hamiltonian.

Optional arguments:

- q=[0,0,0], energies, delta, nk: as above

- RPA=True: dress with the random-phase approximation; `False` for the bare response

h.get_rpa_kernel_poles()

Compute the poles of the generic RPA kernel $1 - V(q)\chi(q, \omega)$: the frequencies of the collective modes/instabilities of the interacting response.

Optional arguments:

- V=None (required): the interaction; a `ValueError` is raised if not given. Either a plain matrix (q-independent, onsite-only) or a real-space hopping dict/`MultiHopping {(n1,n2,n3): matrix}` for an interaction with support beyond the onsite cell, Fourier-transformed to $V(q)$ at this call's `q` (see "Interactions beyond onsite")
- A=None, B=None, q=[0,0,0], energies, delta, nk: as in `get_chi`

Returns an `(npoles,2)` array: pole frequency and its (signed) residual imaginary part – filter on its magnitude, not its raw value, to keep only sharp/well-defined modes – one row per collective mode found, sorted by frequency.

h.get_magnon_bands()

Compute the magnon bands: the poles of the full spin RPA kernel (the same S_x, S_y, S_z channel as `get_spinchi_full`/`get_iets_ldos`, with the interaction taken automatically from the mean-field `h.V` – which can have neighbor-shell, not just onsite, support), scanned along a q-path.

Optional arguments:

- qpath=None, nq=20: the q-path (default path of the geometry) and number of q-points
- energies, delta, nk: as above

Returns `(qs,ws,gammas)`, three flat 1D arrays of equal length: `qs` the integer q-point index along the path, `ws` the pole frequency, `gammas` its residual imaginary part.

h.get_densitychi_RPA()

Compute the density (charge) RPA response function for a `V1`/`V2`/`V3`-neighbor-shell (+ onsite `U`, + general `Vr(r)`) density-density interaction, same convention as `Vinteraction`/`VJinteraction`. Unlike `get_spinchi_full`, the interaction is taken directly as parameters, not read from `h.V` – no mean-field convergence is needed first.

Optional arguments:

- V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None: the density-density interaction, built the same way as `Vinteraction`/`VJinteraction`'s

- q=[0,0,0], energies, delta, nk: as in `get_chi`

h.get_plasmon_bands()

Compute the plasmon/charge-order bands: the poles of the density RPA kernel for a `V1`/`V2`/`V3`/`U`/`Vr` neighbor-shell density-density interaction, scanned along a q-path – the charge-channel analog of `get_magnon_bands`.

Optional arguments:

- V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None: as in `get_densitychi_RPA`
- qpath=None, nq=20, energies, delta, nk: as in `get_magnon_bands`

Returns `(qs,ws,gammas)`, same convention as `get_magnon_bands`.

h.get_fermi_surface()

Compute the spectral weight on a 2D k-mesh at a single energy.

Optional arguments:

- e=0.0: energy of the cut
- nk=50: k-points per direction
- delta: broadening
- operator=None: project/weight by an operator (e.g. `"sz"`, `"valley"`, `"unfold"`)

Returns kx, ky and the Fermi-surface weight

h.get_multi_fermi_surface()

Compute the Fermi surface at many energies, writing one file per energy to a `MULTIFERMISURFACE/` folder.

Optional arguments:

- energies=[0.0]: energies to compute
- nk, delta, operator: as in `get_fermi_surface`

h.get_surface_kdos()

Compute the surface and bulk spectral function of a semi-infinite system, from the surface Green's function (renormalization/decimation technique).

Optional arguments:

- kpath: k-point path (auto-generated if not given)
- energies, delta: frequency range, broadening

Returns k, energy, surface spectral weight and bulk spectral weight; also writes `KDOS.OUT`

h.get_qpi()

Compute the quasiparticle-interference map (2D systems only). Writes output to disk (default `MULTIQPI/` folder plus `DOS.OUT`) rather than returning arrays.

Optional arguments:

- energies, nk, delta: as above
- mode="response": `"pm"` ("poor man's", autoconvolves the actual k-resolved spectral weight – the physical QPI of a real scatterer) or `"response"` (cheaper Lindhard-like joint-DOS convolution of the clean bands)
- nunfold=1: unfold the QPI of a defect embedded in an `nunfoldxnunfold` supercell back onto the primitive Brillouin zone

h.get_qpi_impurity()

Compute quasiparticle interference by placing real-space impurities in a supercell, computing the real-space LDOS with ARPACK partial diagonalization, and Fourier transforming it directly (2D systems only). Returns `(r,ldos_r,q,qpi_q)`.

Optional arguments:

- nsuper=10: supercell size (scalar or `(n1,n2)`)
- impurities=[]: list of dicts, each `{"position"|"index": ..., "onsite": v}` or `{"position"|"index": ..., "vacancy": True}`
- energies=0.0, delta, nk: as above
- num_waves=20: starting number of ARPACK eigenstates computed nearest the requested energies – grown automatically as needed until the diagonalization both reaches `margin` (default 5.0) times `delta` past every requested energy and never stops in the middle of a degenerate manifold (summing over a partial degenerate manifold isn't basis-independent, which otherwise makes the result depend on ARPACK's starting vector – common on symmetric lattices like honeycomb, which have large exact degeneracies at high-symmetry k-points). Picking it too small just costs extra ARPACK calls to grow from, not correctness
- write=True, output_folder="QPI_IMPURITY": also write the MULTIQPI-style disk output

h.get_chern()

Return Chern number of the Hamiltonian.

Optional arguments: - nk=20: number of kpoints - integration="grid": how the Brillouin-zone integral is evaluated. "grid" (default) sums the Berry curvature

over a uniform nk x nk mesh; "qtci" integrates it by quantics tensor cross interpolation plus Gauss-Kronrod quadrature, sampling adaptively instead of uniformly – useful when the curvature is sharply peaked. See "Tensor-cross-interpolation (qtci) integration"

h.get_berry_curvature()

Return the Berry curvature of the occupied bands as a map over the Brillouin zone, `(kx,ky,berry)` – three flat arrays, so it goes straight into a `plt.scatter(kx,ky,c=berry)` or, after reshaping to `(nk,nk)`, into a `contourf`. This is the same curvature that `h.get_chern()` integrates.

Optional arguments:

- nk=100: linear k-point density of the map (the map has `nk*nk` points)
- reciprocal=True: return `kx,ky` in Cartesian reciprocal coordinates, which is what you want to plot a hexagonal Brillouin zone undistorted. Pass `reciprocal=False` for fractional coordinates instead, in which the curvature integrates to the Chern number directly: with `nsuper=1` the map covers `[-1,1)` along both fractional directions, i.e. four Brillouin zones, so `np.sum(berry)*(2/nk)**2/(2*np.pi)` comes out at four times `h.get_chern()`
- mode="Wilson": how the curvature is evaluated. `"Wilson"` uses the Fukui-Hatsugai-Suzuki plaquette construction; `"Green"` uses the Green's-function Kubo formula and is selected automatically when `operator` is given
- operator=None: restrict the curvature to a subspace, e.g. `"valley"` for a valley-resolved curvature (see "Berry curvature operator")
- nsuper=1: extend the map over this many Brillouin zones
- kpath: compute along a k-path instead of over a 2D grid
- delta=0.001: broadening used by the Green's-function mode

h.get_quantum_geometric_tensor()

Return the (multiband/multiorbital) quantum geometric tensor at a single k-point, see "Quantum geometric tensor (multiorbital/multiband)".

Optional arguments: - k=[0.,0.,0.]: k-point - occ_idxs=None: band indices of the chosen subspace (default: the bands with E<0, the same Fermi-level convention `h.get_chern()` uses, so this tracks `h.shift_fermi(...)`) - non_abelian=False: if True, return the full band-pair-resolved tensor instead of its trace over the subspace - degeneracy_tol=1e-10: energy tolerance used to detect a degeneracy between the chosen subspace and its complement (raises `ValueError`)

h.get_quantum_metric()

Same arguments as `h.get_quantum_geometric_tensor()`, but returns only the quantum metric (symmetric part of the tensor).

h.get_wannier_hamiltonian()

Wannierize a fixed range of bands and return the resulting real-space Hamiltonian.

Arguments:

- bands = [a,b]: first and last band to Wannierize (0-indexed, both ends inclusive)

Optional arguments:

- nk=12: k-points per periodic direction for the wannierization mesh
- symmetries=None: `"auto"` to auto-detect and enforce the point group, or an explicit list of `symmetrytk.pointgroup.SymmetryOperation`

Returns a new, smaller Hamiltonian; `.wannier_centres`, `.wannier_spreads` and `.wannier_spread_total` hold the Wannier-function geometry

h.get_szsz_mean_field_hamiltonian()

Self-consistent Hartree-Fock mean field for a $J_z \sum S_i^z S_j^z$ spin-spin exchange interaction (see "Spin-spin exchange interactions"). $J > 0$ is antiferromagnetic, $J < 0$ ferromagnetic.

Optional arguments:

- J1, J2, J3 = 0.: first/second/third-neighbor J_z couplings
- Jr=None: general distance-dependent coupling function, as `Vr` for `get_mean_field_hamiltonian`
- filling, mf, nk, maxerror, mix, constrains: as in `get_mean_field_hamiltonian`
- return_total_energy=False: also return the total energy

Also works on BdG (Nambu) Hamiltonians, decoupling both the normal and anomalous (pairing) channels (same generic dispatch `get_mean_field_hamiltonian` already uses).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

h.get_sxsx_mean_field_hamiltonian() / h.get_sysy_mean_field_hamiltonian()

Same as `get_szsz_mean_field_hamiltonian()`, for a $S_i^x S_j^x$ / $S_i^y S_j^y$ interaction instead, implemented by rotating the problem so that x (or y) becomes the computational z axis, solving there, and rotating the converged Hamiltonian back. Also works on BdG Hamiltonians: `rotate_spin.global_spin_rotation` already rotates the Nambu-doubled Hamiltonian correctly with no changes needed.

h.get_exchange_mean_field_hamiltonian()

Self-consistent anisotropic exchange mean field, combining $J_x S_i^x S_j^x + J_y S_i^y S_j^y + J_z S_i^z S_j^z$ in a single SCF loop.

Optional arguments:

- Jx1, Jx2, Jx3, Jy1, Jy2, Jy3, Jz1, Jz2, Jz3 = 0.: first/second/third-neighbor couplings for each axis
- Jxr, Jyr, Jzr=None: general distance-dependent couplings, one per axis
- mf, filling, nk, maxerror, mix, constrains: as above (only `integration="ed"` and the plain-mixing solver are supported)

Also works on BdG Hamiltonians, with the same full normal-plus-anomalous decoupling as `get_combined_mean_field_hamiltonian`'s density-density channels – exchange can itself induce superconducting pairing (e.g. an antiferromagnetic isotropic J alone, seeded with a random guess, can spontaneously decouple into a purely superconducting state).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

h.get_combined_mean_field_hamiltonian()

Self-consistent mean field combining density-density interactions (onsite U, $V_1/V_2/V_3/V_r$ neighbor-shell) with spin-spin exchange in a single SCF loop.

Optional arguments:

- U, V1, V2, V3, Vr: as in `get_mean_field_hamiltonian`
- J1, J2, J3 = 0.: isotropic Heisenberg exchange for the first/second/third-neighbor shells (same shell convention as V1/V2/V3)
- Jr=None: general distance-dependent isotropic exchange function, as `Vr`
- J1x, J1y, J1z = 0.: optional anisotropic correction added to J1 on the first-neighbor shell only (e.g. the effective first-neighbor Jz coupling is J1+J1z); second/third neighbors stay purely isotropic

On a BdG Hamiltonian, $U/V_1/V_2/V_3/V_r$ keep the full normal+anomalous (pairing) treatment (identical to `get_mean_field_hamiltonian`), while the exchange (J) channels are decoupled in the normal sector only – so a state with both magnetic and superconducting order requires an attractive V, not J, to seed the pairing (see the example above). - mf, filling, nk, maxerror, mix, constrains: as above (only the plain-mixing solver is supported) - integration="ed": computes the density matrix each SCF iteration by exact diagonalization. `"kpm"` instead gets it through a per-k Chebyshev-moment (Kernel Polynomial Method) expansion, never diagonalizing the Bloch Hamiltonian $H(k)$ – for large/sparse systems (e.g. a big 0D flake, where the unit cell itself is too large to diagonalize or even hold as a dense matrix) where per-iteration exact diagonalization is the bottleneck; only supported for a normal-state (non-BdG) Hamiltonian. With `"kpm"`: `scale=None` sets the KPM energy rescaling (estimated automatically if not given), `npol` the number of Chebyshev moments, `ne` the number

of energies sampled in the occupied window, `cores` the number of parallel workers across k-points; all four are unused for `"ed"`. Also reachable through `h.get_mean_field_hamiltonian(integration="kpm",...)`, which dispatches to this function automatically for a spinful `h`.

Performance caveat (measured, not just theoretical): at small/moderate system sizes (order 100-500 sites) `"kpm"` is currently much *slower* per SCF iteration than `"ed"` – roughly 50-60x slower, measured on a 98-site honeycomb Hubbard system – because dense exact diagonalization via LAPACK is extremely fast at that scale regardless of algorithmic complexity, while this KPM implementation still pays real per-orbital and per-matrix-element overhead (see `kpmtk.densitymatrix_kpm._dm_kpm_from_needed`'s and `get_fermi4filling_kpm`'s docstrings for exactly where). It should in principle win for a large/sparse enough system, but that crossover was not reached in the sizes tested. Only use `"kpm"` after confirming it is actually faster for your system. - `use_jax=True, solver="newton"`: solve the same SCF fixed point $x = f(x)$ (x the mean-field parameters, f one SCF iteration) a different way – instead of the default plain-mixing loop above, build a JAX-differentiable version of f and solve it with a JAX-derivative-based root-finder. `solver="newton"` (the default once `use_jax=True`) uses `jax.jacfwd` for the exact Jacobian; `"newton_krylov"` is the matrix-free variant (`jax.jvp` Jacobian-vector products + GMRES), which scales to larger systems than the dense-Jacobian `"newton"` (`gmres_tol=1e-6`, `gmres_restart=20` tune its linear solve); `"fsolve"` wraps `scipy.optimize.fsolve`/MINPACK with the same `jax.jacfwd` Jacobian as `fprime`; `"linear_mixing"` is plain linear mixing routed through the same machinery, for comparison; `"error_gradient"` instead minimizes the squared SCF residual $\|f(x) - x\|^2$ as a proper nonlinear least-squares problem, via matrix-free Levenberg-Marquardt (`jax.jvp`/`jax.vjp` Jacobian-vector and Jacobian-transpose-vector products of the residual + `scipy`'s `lsqr` for each damped LM subproblem) – not the physical free energy directly (see `scftk.vjinteraction_jax`'s module docstring for why that alternative was tried and abandoned: the physical SCF solution turned out to be a saddle point, not a minimum, of the free-energy functional). `"error_gradient"` scales per-iteration like `"newton_krylov"`/ `"linear_mixing"` (no dense Jacobian), and matches `"newton"`/ `"newton_krylov"` well from small systems up through at least ~60 orbitals (see `scftk.vjinteraction_jax`'s module docstring for measured numbers), but as a local method it can in principle still stall short of `maxerror` on a sufficiently hard landscape – always check `.converged`. `"broyden_mixing"` is a black-box mixing scheme rather than a root-finder/gradient method (regularized, limited-memory multisecant Broyden mixing, Marks & Luke arXiv:0801.3098 – see `scftk.broydenmixing`'s module docstring); it only ever evaluates f itself, so it plugs into the same solver dispatch with no Jacobian/gradient machinery of its own, and is also reachable from the plain (non-jax) engine as `solver="broyden_mixing"` (alongside the existing `"broyden1"`/`"krylov"`/`"anderson"`/`"linear"` `scipy.optimize` wrappers in `scftk.densitydensity.generic_densitydensity`). Restricted

to a normal-state (non-BdG) Hamiltonian, dense exact diagonalization only (no `integration="kpm"`), and no `constrains`; needs the optional `jax` extra (`pip install pyqula[jax]`). See `scftk.vjinteraction_jax`'s module docstring for how this reuses (unmodified) the solver infrastructure `scftk.densitydensity_jax` already built for the simpler `Vinteraction` (V/U-only) case:

```
hmf = h.get_combined_mean_field_hamiltonian(U=4.0,J1=-0.5,filling=0.5,
        use_jax=True,solver="newton") # JAX-derivative-based SCF solver
```

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

`filling` also accepts a per-SITE array (length `len(h.geometry.r)`, same 0-to-1-fraction-of-2-orbital-capacity convention as the scalar case) instead of only a single lattice-averaged value, enforcing $\langle n_i \rangle$ = `filling[i]` at every site independently via a per-site Lagrange multiplier (warm-started and co-converged with the mean field across the same SCF loop, one diagonalization per outer iteration – not solved to tight tolerance every iteration, since a per-site potential generally changes the eigenvectors too, unlike a scalar Fermi shift). `scf.local_occupation` and `scf.lam`/`h.fermi` (now the converged per-site array) expose the diagnostics; `scf.converged` implies the per-site constraint converged to within `maxerror`, not only the mean field. Only supported for a normal-state (non-BdG), `integration="ed"` Hamiltonian with `mu=None` (the default). This is the mechanism `SpinonHamiltonian` (see "Abrikosov-pseudofermion (spinon) mean field for Heisenberg models") builds on to enforce exactly one auxiliary fermion per site.

SpinonHamiltonian(g)

Abrikosov-pseudofermion (RVB) mean-field Hamiltonian for a spin-$\frac{1}{2}$ Heisenberg model on geometry `g` – see "Abrikosov-pseudofermion (spinon) mean field for Heisenberg models" above. `from pyqula.spinon import SpinonHamiltonian`; built with zero bare hopping, couplings supplied through `get_mean_field_hamiltonian`'s usual `J1`/`J2`/`J3`/`Jr`/`J1x`/ `J1y`/`J1z` kwargs.

- `h.get_mean_field_hamiltonian(J1=...,nk=...,...)`: same SCF kwargs as `get_combined_mean_field_hamiltonian` above, except `filling` cannot be passed (always exactly one fermion/site, enforced site-by-site). Returns the converged Hamiltonian (or `None` if the SCF did not converge), with two extra diagnostic attributes:
  - `h2.local_occupation`: converged $\langle n_i \rangle$ per site (electron-count convention, 0 to 2 – target is exactly 1.0)
  - `h2.constraint_lambda`: converged per-site Lagrange multiplier (local chemical potential)

KondoLatticeHamiltonian(hc)

Abrikosov-pseudofermion (Read-Newns) mean-field Hamiltonian for the Kondo lattice / periodic Anderson model built from a conduction-electron Hamiltonian `hc` – see "Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice" above. `from pyqula.kondolattice import KondoLatticeHamiltonian`; fuses a second, zero-bare-hopping f-sublattice onto `hc`'s geometry, with the Kondo coupling supplied through `get_mean_field_hamiltonian`'s `J` kwarg.

- `h.get_mean_field_hamiltonian(J=...,filling=...,mf=(V,lam),nk=...,...)`: self-consistently solves for the hybridization and the local constraint's Lagrange multiplier. Returns the converged Hamiltonian (or `None` if the SCF did not converge), with three extra diagnostic attributes:
  - `h2.local_occupation`: converged $\langle n_f \rangle$ per localized site (target is exactly 1.0)
  - `h2.hybridization`: converged V_j per localized site
  - `h2.constraint_lambda`: converged per-site Lagrange multiplier

GrapheneGeometry(g)

`Geometry` subclass wrapping a graphene multilayer geometry `g` (bilayer, twisted bilayer, twisted trilayer, ...), adding a `.relax()` method – see "Twisted bilayer graphene structural relaxation" above. `from pyqula.graphenetk.geometry import GrapheneGeometry`; `g` must have `has_sublattice=True` (raises `ValueError` otherwise).

- `g.relax(nrep=1,maxiter=500,verbose=False,layer_pairs=None,gsfe_coeffs=...,elastic_coeff` minimizes the GSFE (interlayer) + elastic (intralayer) energy over an in-plane displacement field and returns a new, relaxed `GrapheneGeometry`. `layer_pairs` selects which (lower,upper) pairs of layers (ordered by z) get an interlayer GSFE term, defaulting to all adjacent pairs; `gsfe_coeffs`/`elastic_coeffs` override the graphene Table 1 constants of `pyqula.graphenetk.gsfe.GRAPHENE_GSFE` / `pyqula.graphenetk.elastic.GRAPHENE_ELASTIC`.

GrapheneHamiltonian(geometry)

Hamiltonian built from a graphene multilayer `geometry` (typically a `GrapheneGeometry`, relaxed or not) – see "Twisted bilayer graphene structural relaxation" above. `from pyqula.graphenetk.hamiltonian import GrapheneHamiltonian`; defaults to the distance-decaying hoppings of `specialhopping.twisted_matrix` (`ti=0.12,lambi=8.0,lamb=12.0,dl=3.0`, matching `specialhamiltonian.twisted_bilayer_graphene`'s defaults) rather than the generic first-neighbor default, so relaxed (in-plane displaced) positions feed into the electronic structure automatically. Pass `mgenerator=...` to use a different hopping generator instead.

h.get_central_heterostructure()

Build a two-terminal `Heterostructure` using `h` (a finite, 0d Hamiltonian) as the central scattering region, contacted by two semi-infinite 1D chain leads attached at sites `i/j` (see "Transport through an arbitrary finite region" above).

Optional arguments:

- i=0, j=None: 0-indexed sites `h` is contacted at; `j` defaults to the last site
- left=None, right=None: lead Hamiltonians; default to a plain spinless `geometry.chain()`. Give one of them (or `h` itself) nonzero pairing (`add_swave`) for a normal-superconductor junction – at most one of `{h, left, right}` may carry pairing

Returns a `Heterostructure`, so `landauer`, `didv`, `get_dos`, `get_kappa`, etc. all apply unmodified. Only 0d central regions are supported so far (`h.dimensionality>0` raises `NotImplementedError`).

Heterostructure functions and methods

HT.get_dc_current()

Compute the time-averaged (DC) current through a two-terminal junction at a given bias, using the Floquet-Keldysh formalism (see "Multiple Andreev reflection and AC-Josephson current"). Works for any combination of normal/superconducting leads built with `heterostructures.build(h1,h2)`, with or without an explicit `central=` Hamiltonian (the latter is solved by a general dense Floquet inversion, and assumes the bias drops across the junction's rightmost bond).

Arguments:

- voltage: bias voltage

Optional arguments:

- nmax=6, nmax_max=40: initial/maximum number of Floquet sidebands (increased adaptively until convergence; a warning is issued if `nmax_max` is reached before `tol` is satisfied)
- tol=1e-3: relative convergence tolerance used to decide when to stop increasing the sideband count
- temperature=0.: lead temperature
- delta=None: broadening (defaults to `HT.delta`); should be much smaller than the smallest relevant gap for small-gap superconductors

Returns the DC current

HT.get_iv_curve()

Convenience wrapper: `get_dc_current` evaluated over an array of voltages.

Arguments:

- voltages: array of bias voltages

Returns an array of DC currents

HT.didv_curve() / lp.didv_curve()

Convenience wrapper: `didv` evaluated over an array of energies, in parallel – the array-native equivalent of `[ht.didv(energy=e) for e in es]`. Also reachable as `didv(energies=...)` (mutually exclusive with `didv`'s scalar `energy=...`, same convention as `get_kappa`), which dispatches straight here. If `use_aaa=True` and the sweep resolves to the Floquet-Keldysh method, builds and shares one AAA self-energy interpolant across the whole sweep instead of each energy independently building (and discarding) its own – the `didv` counterpart to `get_iv_curve`'s own sharing for `get_dc_current`.

Arguments:

- energies: array of bias energies
- any keyword argument accepted by `didv` (`method`, `delta`, `use_aaa`, `nmax_max`, `temp`, ...)

Returns an array of dI/dV values

HT.get_kappa()

Compute the superconducting/normal conductance power-law-ratio "kappa" diagnostic (also available as `LocalProbe.get_kappa()`, see "Multiple Andreev reflection and AC-Josephson current").

Optional arguments:

- energy=0.0: energy at which to evaluate kappa (returns a scalar)
- energies: array of energies to evaluate at once instead (returns an array); mutually exclusive with `energy`
- temp=0.: temperature; `0.` (the default) uses the original zero-temperature `get_kappa_ratio` path, a nonzero value thermally averages each conductance entering the power-law fit (`didv(temp=...)`) and, for whichever branch is actually superconducting, shares one lead self-energy interpolant across the whole coupling/energy/thermal sweep instead of rebuilding it per call
- T=1e-2: reference coupling the power-law exponent is extracted around

Returns kappa (a scalar, or an array matching `energies`)

At `temp=0.` (the default), kappa is `d(log G)/d(log T)`: how steeply the conductance scales with the probe-sample coupling. The `get_kappa_ratio` path above estimates this by sampling the conductance at two nearby couplings (0.9T and 1.1T) and fitting a secant slope through them. For a `LocalProbe` (always 1D) whose probe lead is not itself superconducting – so `didv` uses the ordinary BdG

scattering-matrix formula, not Floquet-Keldysh – this is now computed exactly instead: neither lead self-energy depends on the coupling `T` (only the coupling block of the small matrix that gets inverted does), so the self-energies are solved once and the coupling-dependent tail is differentiated exactly with `jax.grad` (`transporttk.kappa_jax`), rather than approximated by a secant. This is the default automatically whenever it applies (falling back to the secant otherwise, e.g. when `jax` isn't installed, at finite `temp`, for a superconducting probe, for a spinless-Nambu system, or for a `Heterostructure`); it also cross-checks its own result against the reference `get_smatrix` formula once per call and falls back if they disagree beyond a tight tolerance (guarding against the rare case where `unitarize.check_and_fix`'s unitarity correction on the reference path would have mattered). Benchmarked on `examples/transport/localprobe_kappa_1D`, it matches the secant to within its own finite-difference bias (~1e-3 out of O(1) values) while running faster (~1.2-1.8x depending on system size, after accounting for that cross-check). Both this path and the secant fallback also now solve each lead self-energy once per coupling sweep instead of once per coupling point (self-energies never depend on `T`, only the tiny coupling block that gets inverted does), which is what dominates the speedup for systems where the self-energy solve itself (e.g. a 2D sample's bulk Green's function), not the small coupling-dependent tail, is the expensive part.

SpinModel functions and methods

sm.add_heisenberg()

Add shell-based exchange couplings to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- Jij=None: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor exchange; passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too (`None` is that constructor's own default, first-neighbor hopping)
- Jm=[1.,1.,1.]: diagonal (J_x, J_y, J_z) weights, applied on top of `Jij`; use unequal values for XXZ/Ising-like anisotropy

Mutates `sm` in place, no return value

sm.add_field()

Add a uniform Zeeman field to every site's `sm.b`. Repeated calls accumulate (not overwrite).

Arguments:

- v: 3-vector field, e.g. `[0.,0.,1.]`

sm.add_tensor() / sm.add_tensor_2d()

Add a general pairwise tensor coupling J_{ij} from a function `fun(r1,r2) -> 3x3 matrix` (see `generating_functions` below), rather than the fixed-form couplings `add_heisenberg` builds. `add_tensor` only considers pairs within the home cell; `add_tensor_2d` additionally sums periodic images `ia*a1 + ja*a2` for `ia,ja` in `[-ncells,ncells]`, and can rotate the coupling tensor of each image about z by `vspiral[0]*ia + vspiral[1]*ja` (in units of π) – a way to embed a spin-spiral wavevector directly into the exchange tensor.

Arguments:

- fun: coupling function, e.g. one returned by `generating_functions`

Optional arguments (`add_tensor_2d` only):

- ncells=1: number of periodic images to sum on each side, along each lattice vector
- vspiral=[0.,0.]: per-image in-plane rotation angle coefficients, see above

Mutates `sm` in place, no return value

classicalspin.generating_functions()

Factory returning a `fun(r1,r2) -> 3x3 matrix` two-point coupling function for a standard exchange form, for use with `add_tensor`/`add_tensor_2d`.

Optional arguments:

- name="Heisenberg": one of `"Heisenberg"` (isotropic, cutoff `fc(distance)`), `"Linear"` (dipolar $1/r^3$ tensor), `"RKKYTI"` (RKKY on a topological-insulator surface, PRB 81 233405), `"ZZ"` (Ising, $S_i^z S_j^z$ only), `"XYZ"` (diagonal, anisotropic weights `v`), `"DM"` (Dzyaloshinskii-Moriya, with `v` the intermediate-ion/mirror direction, or a function of the bond vector)
- J=1.0: overall coupling strength
- v=[0.,0.,1.]: form-dependent vector (or vector-valued function of the bond vector), see above
- fc=None: distance-dependent cutoff/envelope, e.g. restricting `"Heisenberg"`/`"ZZ"` to first neighbors (`0.9 < d < 1.1`, the default)
- fdiff=lambda x,y: x-y: how the bond vector is computed from `(r1,r2)`; override for e.g. a minimum-image convention
- fr=None: extra rotation matrix `fr(r1,r2)` applied on top of the base coupling (defaults to the identity)

Returns the coupling function

sm.minimize_energy()

Multistart local minimization of the classical energy over every spin's (θ, ϕ) angles, using `scipy.optimize.minimize` with the gradient from `jax.grad` autodiff of the energy (or, if `calle` is given, gradient-free Powell). Only the ground state

at Γ is found – there are no twisted boundary conditions, so an incommensurate (e.g. spiral) texture needs an explicit supercell that fits it.

Optional arguments:

- theta0=None, phi0=None: initial angles; each try re-randomizes them in $[0, \pi]/[0, 2\pi]$ when left as `None` (the default), which is what makes `tries` explore different basins – passing explicit arrays instead starts every try from the same point, since the optimizer itself is deterministic, so `tries>1` is only useful with the default
- tries=10: number of independent minimizations; the lowest-energy one is kept
- calle=None: optional extra function `calle(sm) -> float` added to the energy during minimization, e.g. a penalty favoring a particular texture

Updates `sm.theta`, `sm.phi` and `sm.magnetization` in place to the best try found, and returns `(theta,phi)`

sm.get_energy() / sm.energy()

Evaluate the total energy $\sum_i \vec{b}_i \cdot \vec{S}_i + \sum_{ij} \vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ of the current `sm.theta`/`sm.phi` snapshot. Returns a scalar.

sm.get_local_energy()

Per-site breakdown of the current snapshot's energy: each site's own field term plus half of every exchange term it takes part in (as with `LatticeGas.get_local_energy()`, each bond's energy is split evenly between its two endpoints), so the values sum exactly to `sm.get_energy()`.

Returns an array over sites

sm.get_magnetization()

Convert the current `sm.theta`/`sm.phi` angles to Cartesian components.

Returns `(mx,my,mz)`, arrays over sites

align.most_perpendicular_vector() / align.most_perp_basis()

(`classicalspintk.align`) Given a set of vectors (typically a `SpinModel`'s magnetization), find the direction most nearly perpendicular to all of them, and use it to build a rotated basis in which that direction is the new z axis – i.e. the vectors end up mostly in the new xy plane. Useful for plotting a magnetization texture (e.g. a skyrmion or spiral) as an in-plane quiver plot when it isn't already aligned with a coordinate axis; see `examples/classicalspin/perpendicular/main.py`.

Returns a perpendicular unit vector, or (for `most_perp_basis`) the input vectors expressed in the rotated basis

LatticeGas functions and methods

lg.add_interaction()

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- Jij: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor $J_{ij} n_i n_j$ repulsion (or attraction, for negative values); passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Mutates `lg` in place, no return value

lg.set_filling()

Reset the occupation array `lg.den` to a new random configuration with a given filling fraction, discarding the current snapshot (e.g. to re-seed a fresh anneal, see `examples/latticegas/optimize/main.py`).

Arguments:

- filling: fraction of sites occupied (rounded to the nearest integer site count)

lg.get_energy()

Evaluate the total energy $\sum_i \mu_i n_i + \sum_{ij} J_{ij} n_i n_j$ of the current occupation snapshot `lg.den`. Returns a scalar.

lg.optimize_energy()

Anneal `lg.den` towards a low-energy configuration with a Metropolis discrete-swap optimizer: at each step, 1-3 random occupied/empty site pairs are swapped (preserving the total filling) and accepted unconditionally if the energy doesn't increase, or with probability $e^{-\Delta E/T}$ otherwise. Energy changes are tracked incrementally per swap rather than by recomputing the full energy from scratch, so cost per step scales with each site's number of interaction neighbors, not with the total number of interaction terms in the system.

Optional arguments:

- temp=0.1: Metropolis temperature (higher accepts more uphill moves; anneal by calling this repeatedly with decreasing `temp`, or see `lg.anneal()` below). `temp=0` runs zero-temperature (greedy) dynamics: an uphill move is never accepted, equivalent to the $T \to 0$ limit without the floating-point division by zero that would otherwise imply
- ntries=1e5: number of swap attempts
- resync_every=1000: how often (in swap attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking

- patience=None: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)
- checkpoint_at=None: an int or iterable of ints; captures a copy of `lg.den` after that many attempts (1-indexed) into `lg.checkpoints` (a dict `step -> den` snapshot), independent of the final configuration – e.g. to inspect or animate how the configuration evolves partway through a run

Overwrites `lg.den` with the final configuration and returns the array of energies recorded at each attempt (whether or not it was accepted)

lg.anneal()

Simulated annealing over a decreasing temperature schedule: calls `lg.optimize_energy()` once per temperature in `temps`, keeping the best configuration seen across the whole schedule (a single high-temperature step's Metropolis walk can wander back up in energy by its end, so the last step's final state is not necessarily the best one found).

Optional arguments:

- temps=None: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from 2.0 down to 0.05 (any entry, including 0, is passed straight through to `lg.optimize_energy()`)
- ntries=1e4: number of swap attempts per temperature
- checkpoint_at=None: an int or iterable of ints; like `lg.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule (e.g. step 120 is the 20th attempt of the 3rd temperature stage if `ntries=100`), so a snapshot can be recovered after any number of annealing steps, not just the final/best configuration
- any other keyword accepted by `lg.optimize_energy()` (e.g. `patience`, `resync_every`), applied at every temperature

Overwrites `lg.den` with the best configuration found and returns the concatenated energy trajectory across all temperatures

lg.optimize_energy_multistart()

Run `nstart` independent anneals from independent random seeds at the current filling, and keep the lowest-energy result – reduces the risk of a single anneal settling into a metastable configuration. Each restart is a full `optimize_discrete` run, farmed out with `parallel.pcall`; whether that runs in parallel depends on `parallel.set_cores()`, same as every other `pcall` call site in this package (serial by default).

Optional arguments:

- nstart=10: number of independent restarts

- any other keyword accepted by `lg.optimize_energy()` (e.g. `temp`, `ntries`, `patience`), applied identically to every restart

Overwrites `lg.den` with the best configuration found and returns its energy (a scalar)

lg.optimize_grand_canonical()

Grand-canonical Metropolis sampling/annealing: instead of swapping pairs at fixed filling, single sites are flipped (occupied $\leftrightarrow$ empty) and accepted/rejected the usual Metropolis way, so the total filling fluctuates under `lg.mu` rather than being conserved. This is the standard lattice-gas MC move set, useful for scanning a phase diagram vs. chemical potential, or for equilibrium sampling at one fixed temperature (see `latticegas.get_specific_heat()`/`get_susceptibility()` below). Unlike `lg.optimize_energy()`, `lg.den` does not need 2 distinct starting values – it can start uniformly empty or full.

Optional arguments: same as `lg.optimize_energy()` (`temp`, `ntries`, `resync_every`; no `patience`)

Overwrites `lg.den` with the final configuration and returns `(es, ns)`: the energy trajectory and the filling (occupied-site count) trajectory, both arrays of length `ntries`

latticegas.get_specific_heat() / latticegas.get_susceptibility()

Module-level (not `lg.`-prefixed) post-processing functions that estimate equilibrium thermodynamic quantities from a trajectory sampled at one *fixed* temperature (e.g. from `lg.optimize_energy()` or `lg.optimize_grand_canonical()` called with a constant `temp`, not annealed) – `get_specific_heat(es, temp, burn=0.2)` returns $C = \mathrm{Var}(E)/T^2$ from an energy trajectory, and `get_susceptibility(ns, temp, burn=0.2)` returns the particle-number susceptibility $\mathrm{d}N/\mathrm{d}\mu = \mathrm{Var}(N)/T$ from a filling trajectory (only meaningful for the grand-canonical `ns`, since `lg.optimize_energy()`'s filling is constant by construction). `burn` discards that leading fraction of the trajectory as equilibration before computing the variance.

lg.add_tensor()

Add a custom coupling $J_{ij} = \mathrm{fun}(r_i, r_j)$ between every pair of sites, for interactions beyond `add_interaction()`'s fixed neighbor shells – e.g. a screened or dipolar $1/r^n$ form. Scalar analog of `classicalspin.SpinModel.add_tensor` (which returns a 3x3 tensor per pair); self-pairs are skipped, and pairs where `fun` evaluates to (near) zero are dropped.

Mutates `lg` in place, no return value

lg.regroup()

Merge duplicate interaction-pair entries accumulated from repeated `add_interaction()`/`add_tensor()` calls (e.g. overlapping neighbor shells added twice), summing their couplings – pure performance cleanup, doesn't change `lg.get_energy()`.

Mutates `lg` in place, no return value

lg.write() / lg.read()

Save/load the current occupation snapshot `lg.den` to/from a text file, reusing `Geometry.write_profile()` – the same checkpoint pattern as `classicalspin.SpinModel.write()`/`load_magnetism()`. `write()` forces `nrep=1` (no periodic replication) by default so `read()` round-trips exactly regardless of `lg.geometry.dimensionality`; `read()` raises `ValueError` if the file's site count doesn't match `lg.nsites`.

Optional arguments:

- name="DENSITY.OUT": file path

lg.get_local_energy() / lg.get_local_mu()

Per-site breakdown of the current snapshot's energy. `get_local_energy()` returns each site's own contribution $\mu_i n_i + \sum_j J_{ij} n_i n_j$, summed over its interaction neighbors j (`lg.get_energy()` itself counts every bond twice, once from each endpoint, so the values here sum exactly to `lg.get_energy()`, not to half of it); `get_local_mu()` instead evaluates that same expression with site `i` forced occupied, i.e. the energy cost/gain of occupying site `i` given its neighbors' current state.

Optional arguments:

- normalize=False: divide each site's value by the total coupling weight of its own interaction terms

Returns an array over sites

lg.get_correlator()

Neighbor-shell density-density correlator of the current snapshot `lg.den`, useful for detecting ordered ground states (e.g. after `optimize_energy`). Thin wrapper around `statphystk.correlator.get_nnc`; see its docstring for `n`/`normalized` options.

Returns `(distances, correlators)`, arrays of matching length

lg.get_structure_factor()

Reciprocal-space structure factor $S(q) = |\sum_i (n_i - \bar{n}) e^{-iq\cdot r_i}|^2/N$ of the current snapshot `lg.den`, evaluated directly on the real-space site positions – the reciprocal-space companion to `lg.get_correlator()`: where the neighbor-shell correlator tells you the ordering length scale, $S(q)$ tells you the ordering wavevector. Subtracting the mean occupation makes $S(q = 0) = 0$ identically, so a peak elsewhere in q is what signals order. Thin wrapper around `statphystk.correlator.get_structure_factor`.

Optional arguments:

- qpath=None: explicit array of q vectors to evaluate; if omitted, a default square grid of `nq`×`nq` points spanning ±`qmax` is used
- nq=60: grid resolution when `qpath` is not given
- qmax=None: half-width of the default grid; defaults to $2\pi/d$ set by the nearest-neighbor spacing d

Returns `(qpath, sq)`: the array of q vectors evaluated and the matching array of $S(q)$ values

LatticeIsing functions and methods

li.add_interaction()

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- Jij: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor $-J_{ij} s_i s_j$ exchange; positive is ferromagnetic (opposite sign convention from `LatticeGas.add_interaction()`). Passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Mutates `li` in place, no return value

li.add_field()

Add an external (Zeeman-like) field to `li.b`.

Arguments:

- h: a scalar (applied uniformly to every site) or a per-site array

Mutates `li` in place, no return value

li.set_magnetization()

Reset the spin array `li.s` to a new random ±1 configuration with a given average magnetization, discarding the current snapshot.

Arguments:

- m=0.0: target average magnetization in $[-1, 1]$ (rounded to the nearest integer up-spin count)

li.get_energy()

Evaluate the total energy $-\sum_i b_i s_i - \sum_{ij} J_{ij} s_i s_j$ of the current spin snapshot `li.s`. Returns a scalar. Since `li.pairs` lists both directions of every bond, this is twice the usual sum-over-unordered-bonds convention.

li.get_magnetization()

Return mean(s), the average magnetization per site of `li.s` (a scalar in $[-1, 1]$).

li.optimize_energy()

Single-spin-flip Metropolis dynamics – the standard Ising Monte Carlo move set: at each step, one random site is flipped and accepted unconditionally if the energy doesn't increase, or with probability $e^{-\Delta E/T}$ otherwise. Magnetization is *not* conserved (it fluctuates under `li.b`), the spin analog of `LatticeGas.optimize_grand_canonical()`.

Optional arguments:

- temp=1.0: Metropolis temperature; `temp=0` runs zero-temperature (greedy) dynamics
- ntries=1e5: number of flip attempts
- resync_every=1000: how often (in flip attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking
- checkpoint_at=None: an int or iterable of ints; captures a copy of `li.s` after that many attempts (1-indexed) into `li.checkpoints` (a dict `step -> s` snapshot)

No `patience` option: unlike `optimize_conserved()`, this trajectory is meant to be fed to `latticegas.get_specific_heat()`/`get_susceptibility()`, and early truncation would silently bias those variance estimates.

Overwrites `li.s` with the final configuration and returns `(es, ms)`: the energy trajectory and the total-magnetization ($\sum_i s_i$) trajectory, both arrays of length `ntries`

li.optimize_conserved()

Kawasaki spin-exchange dynamics: at each step, one up spin and one down spin are picked at random and swapped, which conserves the total magnetization – the spin analog of `LatticeGas.optimize_energy()` (swap-based, fixed filling). Raises `ValueError` if `li.s` doesn't currently have both $+1$ and -1 present (e.g. after `set_magnetization(1.0)`).

Optional arguments: same as `li.optimize_energy()`, plus:

- patience=None: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)

Overwrites `li.s` with the final configuration and returns the array of energies recorded at each attempt

li.anneal()

Simulated annealing over a decreasing temperature schedule: calls `li.optimize_energy()` once per temperature in `temps`, keeping the best (lowest-energy) configuration seen across the whole schedule.

Optional arguments:

- temps=None: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from 2.0 down to 0.05
- ntries=1e4: number of flip attempts per temperature
- checkpoint_at=None: like `li.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule
- any other keyword accepted by `li.optimize_energy()`, applied at every temperature

Overwrites `li.s` with the best configuration found and returns `(es, ms)`, the concatenated energy and magnetization trajectories across all temperatures

li.optimize_energy_multistart()

Run `nstart` independent flip-based anneals from independent random seeds (same initial magnetization as the current `li.s`) and keep the lowest-energy result. Each restart is a full `optimize_ising` run, farmed out with `parallel.pcall`, mirroring `LatticeGas.optimize_energy_multistart()`.

Optional arguments:

- nstart=10: number of independent restarts
- any other keyword accepted by `li.optimize_energy()` (e.g. `temp`, `ntries`), applied identically to every restart

Overwrites `li.s` with the best configuration found and returns its energy (a scalar)

li.get_local_energy() / li.get_local_field()

Per-site breakdown of the current snapshot. `get_local_energy()` returns each site's own contribution to `li.get_energy()` (values sum exactly to `li.get_energy()`, mirroring `LatticeGas.get_local_energy()`'s $j/2$ correction for the double-counted bonds). `get_local_field()` returns the effective

field $h_i^{\text{eff}} = b_i + 2\sum_k J_{ik} s_k$ seen by each site, defined so that flipping s_i costs exactly $2 s_i h_i^{\text{eff}}$ – the spin analog of `LatticeGas.get_local_mu()`.

Returns an array over sites

li.get_correlator() / li.get_structure_factor()

Spin-spin correlator and reciprocal-space structure factor of the current snapshot `li.s`, reusing `statphystk.correlator.get_nnc/get_structure_factor` directly (both operate on any per-site array, not just 0/1 occupations) – see `LatticeGas.get_correlator()/get_structure_factor()` for the argument reference.

li.add_tensor() / li.regroup()

Same as `LatticeGas.add_tensor()/regroup()`, reusing those functions directly (they only touch `geometry.r/nsites/pairs/j`, none of which differ in meaning between the two models).

Mutates `li` in place, no return value

li.write() / li.read()

Save/load the current spin snapshot `li.s` to/from a text file, reusing `Geometry.write_profile()` – see `LatticeGas.write()/read()`. `read()` rounds to $\{-1, +1\}$ (sign, treating exact 0 as +1) to absorb the text round-trip's floating point noise, and raises `ValueError` if the file's site count doesn't match `li.nsites`.

Optional arguments:

- name="SPIN.OUT": file path